

**Universidad Central “Marta Abreu” de Las Villas**

Facultad de Ingeniería Eléctrica

Departamento de Electrónica y Telecomunicaciones

# **Mejora del sincronismo en redes inalámbricas empleando el protocolo gPTP**

Trabajo de Diploma presentado en opción al título de  
Ingeniero en Telecomunicaciones y Electrónica

**Autor:** [Nombre del autor]

**Tutor:** [Nombre del tutor]

Santa Clara, Cuba  
2025

# Resumen

El sincronismo de tiempo constituye uno de los temas de investigación más estudiados en las redes inalámbricas, especialmente para numerosas aplicaciones en el ámbito del IoT. En consecuencia, se han desarrollado diversos protocolos de sincronismo para satisfacer las demandas de las redes contemporáneas, entre los que sobresale el Protocolo de Tiempo de Precisión Generalizado (gPTP); sin embargo, su eficiencia en el sincronismo se ve gravemente afectada ante la presencia de asimetrías en los canales de comunicación.

La presente investigación tiene como objetivo mitigar el efecto de los retardos asimétricos en el sincronismo en redes inalámbricas empleando una versión mejorada del protocolo gPTP. Para su cumplimiento, se realizó una revisión de las bases teóricas relacionadas con el tema y un análisis sistemático de 15 métodos existentes para mejorar el sincronismo en redes inalámbricas empleando gPTP, abarcando filtros de Kalman, estimación robusta, lógica difusa y aprendizaje computacional.

Como resultado, se propone un enfoque híbrido que combina la corrección determinista de estampas de tiempo de Reinhard Exel con un Filtro de Kalman Adaptativo (AKF) de tres estados para la estimación conjunta de offset, skew y asimetría residual. Se implementó un simulador de eventos discretos del protocolo gPTP en MATLAB/GNU Octave, evaluado mediante simulación Monte Carlo de 3000 ejecuciones.

Las evaluaciones realizadas confirman que la corrección Exel reduce el impacto de las asimetrías en un valor medio de  $50.7 \mu s$ , lo que representa un aumento de la efectividad del sincronismo del 33.31 %. Se proyecta que el método híbrido Exel+AKF propuesto alcance una mejora adicional del 28.8 %, para una reducción total del error del 52.5 % respecto al protocolo sin corrección, con un *overhead* computacional moderado.

**Palabras clave:** sincronismo, redes inalámbricas, protocolo gPTP, asimetrías, PTP, IEEE 802.1AS, Filtro de Kalman Adaptativo.

# Abstract

Time synchronization is one of the most studied research topics in wireless networks, especially for numerous applications in the IoT domain. Consequently, various synchronization protocols have been developed to meet the demands of contemporary networks, among which the Generalized Precision Time Protocol (gPTP) stands out; however, its synchronization efficiency is severely affected by the presence of asymmetries in communication channels.

This research aims to mitigate the effect of asymmetric delays on synchronization in wireless networks using an enhanced version of the gPTP protocol. To achieve this, a review of the theoretical foundations and a systematic analysis of 15 existing methods for improving synchronization in wireless networks using gPTP were conducted, covering Kalman filters, robust estimation, fuzzy logic, and machine learning.

As a result, a hybrid approach is proposed that combines Reinhard Exel's deterministic timestamp correction with a three-state Adaptive Kalman Filter (AKF) for joint estimation of offset, skew, and residual asymmetry. A discrete-event simulator of the gPTP protocol was implemented in MATLAB/GNU Octave and evaluated through 3000-run Monte Carlo simulation.

The evaluations confirm that the Exel correction reduces the impact of asymmetries by an average value of  $50.7 \mu\text{s}$ , representing a 33.31 % increase in synchronization effectiveness. The proposed hybrid Exel+AKF method is projected to achieve an additional 28.8 % improvement, for a total error reduction of 52.5 % compared to the uncorrected protocol, with moderate computational overhead.

**Keywords:** synchronization, wireless networks, gPTP protocol, asymmetries, PTP, IEEE 802.1AS, Adaptive Kalman Filter.

# Agradecimientos

A mis padres, por ser el ejemplo a seguir en la vida y haberme dado todo el apoyo necesario para cumplir este sueño.

A mis hermanos, por depositar toda su confianza en mí y brindarme siempre su ayuda incondicional.

A mi tutor, por su guía, su paciencia y su disponibilidad en todo momento a lo largo de este proceso. Ha sido un placer trabajar bajo su dirección.

A todos aquellos profesores que han sido parte de mi formación académica, por compartir su conocimiento y despertar en mí la pasión por las telecomunicaciones y la investigación.

A mis amigos y compañeros, por los momentos compartidos, el apoyo mutuo y las incontables horas de estudio.

A todos, mi más sincero agradecimiento.

# Índice general

|                                                                                                                                                          |           |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                                                                                                           | <b>1</b>  |
| <b>Abstract</b>                                                                                                                                          | <b>2</b>  |
| <b>Agradecimientos</b>                                                                                                                                   | <b>3</b>  |
| <b>1. Introducción</b>                                                                                                                                   | <b>6</b>  |
| <b>2. Aplicaciones y desafíos del sincronismo en redes inalámbricas. Asimetría en los canales inalámbricos y protocolos de sincronismo</b>               | <b>11</b> |
| 2.1. Sincronismo en redes inalámbricas . . . . .                                                                                                         | 11        |
| 2.2. Aplicaciones del sincronismo en IoT . . . . .                                                                                                       | 13        |
| 2.3. Desafíos del sincronismo en redes inalámbricas . . . . .                                                                                            | 15        |
| 2.3.1. Retardos de propagación variables . . . . .                                                                                                       | 17        |
| 2.3.2. Deriva y ruido de los osciladores . . . . .                                                                                                       | 17        |
| 2.3.3. Asimetrías en los canales de comunicación . . . . .                                                                                               | 17        |
| 2.3.4. Limitaciones de recursos en los dispositivos . . . . .                                                                                            | 18        |
| 2.3.5. Escalabilidad y densidad de la red . . . . .                                                                                                      | 18        |
| 2.3.6. Seguridad . . . . .                                                                                                                               | 18        |
| 2.4. Asimetrías en los canales de comunicación . . . . .                                                                                                 | 18        |
| 2.5. Protocolos de sincronismo . . . . .                                                                                                                 | 19        |
| 2.5.1. Protocolo de Tiempo de Precisión (PTP) . . . . .                                                                                                  | 20        |
| 2.5.2. Protocolo de Tiempo de Precisión Generalizado (gPTP) . . . . .                                                                                    | 21        |
| 2.6. Conclusiones del capítulo . . . . .                                                                                                                 | 24        |
| <b>3. Análisis de métodos y herramientas para mejorar el sincronismo en redes inalámbricas empleando el protocolo gPTP</b>                               | <b>26</b> |
| 3.1. Sistemas de estampado por Software . . . . .                                                                                                        | 26        |
| 3.2. Análisis de los modelos, metodologías y procedimientos existentes para el diseño de protocolos más robustos ante los retardos asimétricos . . . . . | 28        |
| 3.2.1. Método de corrección determinista de estampas de tiempo (Exel) . . . . .                                                                          | 28        |
| 3.2.2. Métodos basados en Filtros de Kalman . . . . .                                                                                                    | 29        |
| 3.2.3. Métodos basados en Estimación Robusta . . . . .                                                                                                   | 31        |
| 3.2.4. Métodos basados en Lógica Difusa . . . . .                                                                                                        | 31        |
| 3.2.5. Métodos basados en Aprendizaje Computacional . . . . .                                                                                            | 32        |
| 3.2.6. Otros métodos relevantes . . . . .                                                                                                                | 32        |
| 3.2.7. Comparación sistemática y selección del método . . . . .                                                                                          | 32        |
| 3.3. Descripción del método seleccionado: Enfoque híbrido Exel + Filtro de Kalman Adaptativo . . . . .                                                   | 33        |

|              |                                                                                                                           |           |
|--------------|---------------------------------------------------------------------------------------------------------------------------|-----------|
| 3.3.1.       | Arquitectura de dos etapas . . . . .                                                                                      | 33        |
| 3.3.2.       | Modelo de espacio de estados . . . . .                                                                                    | 33        |
| 3.3.3.       | Algoritmo del Filtro de Kalman Adaptativo . . . . .                                                                       | 35        |
| 3.3.4.       | Parámetros de diseño . . . . .                                                                                            | 36        |
| 3.3.5.       | Ventajas del enfoque híbrido . . . . .                                                                                    | 36        |
| 3.4.         | Herramientas de implementación: MATLAB y GNU Octave . . . . .                                                             | 36        |
| 3.4.1.       | Archivos-M . . . . .                                                                                                      | 38        |
| 3.4.2.       | Graficado en 2-D . . . . .                                                                                                | 38        |
| 3.4.3.       | Ventajas y desventajas de la programación en MATLAB/Octave . . . . .                                                      | 38        |
| 3.5.         | Conclusiones del capítulo . . . . .                                                                                       | 39        |
| <b>4.</b>    | <b>Implementación de la corrección del protocolo gPTP ante los retardos asimétricos en redes inalámbricas. Resultados</b> | <b>40</b> |
| 4.1.         | Características de la implementación . . . . .                                                                            | 40        |
| 4.1.1.       | Arquitectura general de la simulación . . . . .                                                                           | 40        |
| 4.1.2.       | Implementación de referencia: gPTP + Exel . . . . .                                                                       | 41        |
| 4.1.3.       | Implementación mejorada: gPTP + Exel + AKF . . . . .                                                                      | 41        |
| 4.1.4.       | Simulación Monte Carlo . . . . .                                                                                          | 41        |
| 4.2.         | Análisis de los resultados . . . . .                                                                                      | 43        |
| 4.2.1.       | Resultados de la implementación de referencia (Exel) . . . . .                                                            | 43        |
| 4.2.2.       | Resultados de la implementación mejorada (Exel + AKF) . . . . .                                                           | 48        |
| 4.2.3.       | Comparación global de escenarios . . . . .                                                                                | 48        |
| 4.3.         | Conclusiones del capítulo . . . . .                                                                                       | 55        |
| <b>5.</b>    | <b>Conclusiones</b>                                                                                                       | <b>56</b> |
| <b>6.</b>    | <b>Recomendaciones</b>                                                                                                    | <b>58</b> |
| <b>A. A.</b> | <b>Parámetros de simulación</b>                                                                                           | <b>65</b> |
| <b>B. B.</b> | <b>Código fuente de la implementación</b>                                                                                 | <b>66</b> |
|              | <b>Acrónimos</b>                                                                                                          | <b>67</b> |

# Capítulo 1

## Introducción

El Internet de las Cosas (*Internet of Things*, IoT) ha revolucionado la vida cotidiana al permitir el intercambio de datos sin interrupciones entre varios dispositivos a través de Internet. Sin embargo, ha traído consigo desafíos significativos para las aplicaciones que requieren una temporización sincronizada, como garantizar la secuencia ordenada de los datos o el rendimiento sincronizado de las actividades. Las redes de IoT suelen estar compuestas por entidades con recursos variables, lo que dificulta la implementación de métodos de sincronización de tiempo tradicionales en dispositivos con recursos limitados. Las aplicaciones de IoT cubren una amplia gama de necesidades humanas y vitales, desde entornos inteligentes (ciudades, hogar, transporte, etc.) hasta la salud y la calidad de vida [1].

En este ámbito, las redes inalámbricas de sensores (*Wireless Sensors Network*, WSN) han adquirido un papel importante en una amplia gama de aplicaciones, atribuidas principalmente a su rentabilidad, capacidades funcionales, adaptabilidad y amplia cobertura. Sin embargo, la eficacia de las WSN se ve comprometida por obstrucciones que perjudican la intensidad de la señal y afectan el sincronismo. En consecuencia, la sincronización de reloj es esencial para el funcionamiento de las aplicaciones en redes inalámbricas de sensores, ya que logra mediciones confiables, facilita la coordinación de acciones y garantiza un intercambio de datos fluido entre dispositivos [2, 3]. Las WSN son utilizadas en muchas aplicaciones en las que se requiere una sincronización parcial o completa en la red, como el control de fusión nuclear, la automatización de plantas de fabricación, la comunicación móvil, el diagnóstico de fallos mecánicos, la robótica, el Big Data, el diagnóstico médico y la educación [4, 5].

Los sistemas de control industriales en la era de la Industria 4.0 presentan desafíos significativos desde el punto de vista de la comunicación: baja latencia, confiabilidad y sincronización precisa. En este sentido, las redes sensibles al tiempo (*Time Sensitive Network*, TSN) se han convertido en la principal solución de estos desafíos; por tanto, un requisito estratégico de la sincronización para TSN inalámbricas es lograr que la solución sea autónoma, es decir, que las funcionalidades sean realizadas por la propia red, sin depender de una infraestructura externa [6]. Las TSN se basan en cuatro conceptos claves: sincronización de tiempo precisa, baja latencia garantizada, ultra confiabilidad y gestión de recursos. La tecnología inalámbrica ha aportado nuevas posibilidades a las comunicaciones de red, pero los enlaces inalámbricos introducen falta de fiabilidad, canales y latencias asimétricas, interferencia de canal y distorsión de la señal, lo que dificulta el desarrollo de normas e implementaciones adecuadas para el sincronismo en redes inalámbricas [7].

La sincronización de reloj es uno de los temas de investigación más populares en los sistemas distribuidos de medición y control, ya que en estas redes hay un gran número de nodos controlados por relojes independientes y se requiere que cada nodo tenga una referencia de tiempo común [8]. Una de las limitaciones más acuciantes es la presencia de retardos asimétricos en los canales de comunicación, ya que son una de las principales razones de la inexactitud del proceso de sincronismo. La presencia de asimetrías puede degradar significativamente el rendimiento de los esquemas de estimación del desfase y la desviación del reloj, y desafortunadamente no pueden ser medidas en un sistema de sincronización por pares ni siquiera intercambiando un número infinito de mensajes [9]. Los protocolos de sincronismo tradicionales asumen que los retardos son simétricos; cuando surge una asimetría en el enlace, estos protocolos crean un desfase de reloj significativo. Debido a esta limitación fundamental, los protocolos convencionales no pueden mitigar el desfase creado por una asimetría desconocida [10, 11].

Dentro de los protocolos existentes, el Protocolo de Tiempo de Precisión Generalizado (*Generalized Precision Time Protocol*, gPTP), definido en IEEE 802.1AS [12], destaca por ser uno de los pocos especificados para su uso en redes inalámbricas, además de obtener una eficiencia en sincronización en el orden de los nanosegundos. Sin embargo, asume que el retardo de la red es simétrico; por tanto, ante la presencia de asimetrías se ve afectada su eficiencia, lo que puede provocar errores de sincronismo significativos.

En correspondencia con lo expuesto, se propone el siguiente **problema de investigación**: ¿Cómo mejorar el sincronismo en redes inalámbricas ante la presencia de retardos de propagación asimétricos empleando el protocolo gPTP? A partir de esta problemática se define como **objetivo general**: Mitigar el efecto de los retardos asimétricos en el sincronismo en redes inalámbricas empleando una versión mejorada del protocolo gPTP. Para dar cumplimiento al objetivo general se plantean los **objetivos específicos** siguientes:

1. Realizar una revisión de las bases teóricas relacionadas con el sincronismo en redes inalámbricas, a partir del estudio de la literatura.
2. Analizar las herramientas empleadas y los métodos existentes —incluyendo técnicas novedosas basadas en filtrado de Kalman— para mejorar el sincronismo en redes inalámbricas empleando el protocolo gPTP.
3. Implementar una versión mejorada del protocolo gPTP que incorpore un Filtro de Kalman Adaptativo como segunda etapa de corrección de asimetrías.
4. Evaluar el rendimiento de la implementación mejorada en comparación con la implementación de referencia y el protocolo estándar, a través de distintas métricas de precisión y estabilidad.

La **hipótesis** de la investigación se formula de la siguiente manera: si se emplea una versión del protocolo gPTP que combine la corrección determinista de estampas de tiempo con un filtrado estadístico adaptativo, entonces se logra mitigar los efectos negativos de los retardos de propagación asimétricos con una efectividad superior a la alcanzada por el método de corrección determinista por sí solo, lo que resulta en una mejora significativa adicional en la precisión del sincronismo.

En el desarrollo de la investigación se utilizaron los siguientes **métodos científicos**: analítico-sintético, inductivo-deductivo, estadístico-matemáticos, medición y modelación.



Ciclo R-E-V-T (Research-Execute-Validate-Test)

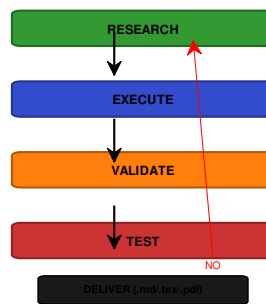


Figura 1.1: Ciclo iterativo Research-Execute-Validate-Test aplicado en el desarrollo de la investigación.

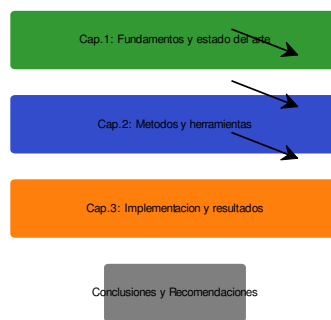


Figura 1.2: Estructura lógica de la tesis.

Para su presentación, este trabajo de diploma cuenta con una estructura lógica compuesta por el resumen de la investigación, el índice, la introducción, y tres capítulos donde se desarrolla la investigación. En el **Capítulo 1** se realiza una revisión bibliográfica que aborda el sincronismo en redes inalámbricas, sus principales aplicaciones en IoT, los desafíos técnicos, el efecto de las asimetrías y una caracterización de los protocolos de sincronización PTP y gPTP. En el **Capítulo 2** se efectúa un análisis sistemático de 15 métodos existentes para mitigar el efecto de los retardos asimétricos, organizados en siete categorías (corrección determinista, filtros de Kalman, estimación robusta, lógica difusa, aprendizaje computacional, filtros de partículas y programación lineal), y se selecciona y describe en detalle un enfoque híbrido que combina la corrección de Exel con un Filtro de Kalman Adaptativo. El **Capítulo 3** describe las características de la implementación desarrollada en MATLAB/GNU Octave y presenta los resultados de la simulación Monte Carlo, comparando el rendimiento del método propuesto con el método de referencia Exel y con el protocolo estándar. Finalmente, se expresan las conclusiones y recomendaciones para trabajos futuros, y se incluyen las referencias bibliográficas según la norma IEEE.

## Capítulo 2

# Aplicaciones y desafíos del sincronismo en redes inalámbricas. Asimetría en los canales inalámbricos y protocolos de sincronismo

### 2.1. Sincronismo en redes inalámbricas

La sincronización de tiempo constituye uno de los servicios más importantes en las redes de telecomunicaciones modernas, ya que permite coordinar el acceso distribuido a los recursos compartidos, comparar las marcas de tiempo establecidas en diferentes ubicaciones y medir el rendimiento de los sistemas distribuidos en tiempo real [4]. El propósito fundamental de la sincronización es el establecimiento de una noción global del tiempo con una precisión predefinida, asegurando desplazamientos máximos acotados entre dos nodos cualesquiera [13].

El sincronismo se define como el proceso de entrega de una «referencia común» a los elementos de red desde una fuente común dentro de una precisión y estabilidad dadas, a fin de controlar precisamente la tasa a la cual las señales digitales se transmiten y procesan a través de dicha red. Desde el Internet de las Cosas (*Internet of Things*, IoT) se imponen restricciones a las aplicaciones que requieren redes sincronizadas en el tiempo para un orden cronológico de la información o la ejecución sincrónica de algunas tareas [14]. A lo largo de los años, la sincronización de tiempo ha ganado relevancia en campos como las telecomunicaciones y la automatización industrial, impulsada por la necesidad de sistemas de comunicación más confiables y eficientes.

La sincronización horaria es un componente esencial de las redes sensibles al tiempo (*Time Sensitive Network*, TSN), en especial de las redes inalámbricas de sensores (*Wireless Sensors Network*, WSN), ya que una amplia gama de aplicaciones potenciales para estas redes requiere una sincronización precisa [15, 16]. Las WSN han adquirido un papel importante en una amplia gama de aplicaciones, atribuidas principalmente a su rentabilidad, capacidades funcionales, adaptabilidad y amplia cobertura. Estas redes son un tipo especial de red donde los dispositivos inalámbricos (generalmente denominados nodos de la red) trabajan juntos para formar espontáneamente una red sin la necesidad de ninguna infraestructura. Los nodos pueden enviar datos, recibirlos o actuar como enrutadores en la red [5].

Según [17], los principales tipos de sincronización de relojes son:

1. **Reloj global:** el Tiempo Universal Coordinado (UTC) es el estándar de tiempo comúnmente utilizado en todo el mundo. Los algoritmos tradicionales de sincronización de Internet utilizan el UTC para mantener esta hora global en todos los sistemas informáticos. Esta precisión del tiempo global es mantenida por un reloj atómico; sin embargo, mantener esta sincronización en las redes de sensores es significativamente más difícil, lo que impone una sobrecarga de comunicación.
2. **Reloj relativo:** esta es la idea del tiempo relativo dentro de la red de sensores. Cada nodo se sincroniza con todos los demás nodos con una información de tiempo determinada, que puede ser diferente de UTC.
3. **Ordenamiento físico:** este modelo se utiliza para ordenar los eventos de los procesos en un sistema que comparte el mismo reloj o tiene una memoria compartida.
4. **Noción relativa del tiempo:** el tiempo también se puede acordar entre nodos sin usar un reloj en tiempo real, sino usando un tiempo lógico. Este modelo no tiene por qué coincidir con el reloj físico, es decir, cuando dos nodos establecen una conexión, este momento se establece como un tiempo lógico «cero» y, a partir de este momento, cada unidad de tiempo se incrementa por ambos nodos, y así se establece un sincronismo.

La sincronización de tiempo tiene como objetivo crear o establecer una base de tiempo común entre los dispositivos o nodos de la red [5, 18, 19], con el fin de poder comparar eventos o sintonizar acciones [20, 21]. Esto es una necesidad para muchas aplicaciones; numerosas investigaciones en torno a la comunicación inalámbrica han demostrado el uso de la sincronización de tiempo en diferentes campos de la automatización industrial, incluida la robótica, las redes inteligentes, el monitoreo mediante redes de sensores de área amplia e incluso en entornos electromagnéticos muy desafiantes [22].

Existen muchos ejemplos de aplicaciones donde la sincronización temporal desempeña un papel fundamental, como los protocolos de Control de Acceso al Medio (*Medium Access Control*, MAC), la monitorización de la salud estructural (*Structural Health Monitoring*, SHM) y la medición inteligente [23]. Las técnicas de control de acceso al medio, como el Acceso Múltiple por División de Tiempo (TDMA), requieren sincronización de tiempo para una programación precisa de los accesos al medio sin colisiones. Además, las WSN tienen recursos energéticos limitados, por lo que se utilizan técnicas de ahorro de energía para reducir el consumo de toda la red. Para que los nodos enciendan y apaguen sus transceptores en los momentos adecuados, se requiere una sincronización precisa entre los nodos de la red [5].

Las técnicas de sincronización tradicionales que se han utilizado en redes cableadas no son adecuadas para redes inalámbricas. Para sincronizar los nodos en este tipo de redes, es necesario alcanzar una noción común del tiempo teniendo en cuenta que los relojes de los dispositivos pueden desviarse por múltiples razones, que los requisitos de tiempo de cada aplicación varían y que los parámetros objetivo a maximizar en la aplicación difieren en cada caso [11].

Los relojes tienen dos fuentes principales de imprecisión: el desfase (*offset*) y la desviación (*skew*); por tanto, estos son los principales objetos de corrección para lograr la sincronización de tiempo [20]. El desfase es la diferencia absoluta entre el valor de un reloj y el valor de otro, mientras que la desviación es la diferencia entre la frecuencia

de los dos relojes. Debido a la desviación del reloj se produce un aumento constante del desfase con el paso del tiempo. Matemáticamente, el tiempo local  $t_u$  en el nodo  $u$  está relacionado con el tiempo global  $t$  (desconocido) por la Ecuación 2.1:

$$t_u = \alpha_u t + \beta_u \quad (2.1)$$

Donde  $\alpha_u$  y  $\beta_u$  son la desviación y el desfase del reloj en el nodo  $u$ , respectivamente. El desfase relativo entre un par de nodos  $u$  y  $v$  en una red se puede medir (hasta algún error) intercambiando mensajes con marca de tiempo entre ellos [16].

Los protocolos de sincronización de relojes se basan en la comunicación de la estación maestra de tiempo (*master clock*) a la estación esclava (*slave*), y pueden ser unidireccionales o bidireccionales. En el primer caso, el maestro actúa como estación emisora y puede enviar señales de tiempo de forma continua o periódica, pero no recibe retroalimentación del esclavo. En el esquema bidireccional, los dispositivos intercambian mensajes con marcas de tiempo, permitiendo al esclavo estimar el desfase respecto al maestro y corregir su reloj local. Este último enfoque es el adoptado por los principales protocolos de sincronización de precisión, como el Protocolo de Tiempo de Precisión (PTP) y su perfil generalizado (gPTP).

En los últimos años, la integración de redes TSN con tecnologías inalámbricas como 5G ha generado un renovado interés en la sincronización de precisión sobre enlaces inalámbricos [24, 25]. La capacidad de 5G para proporcionar conectividad de baja latencia y alta confiabilidad lo convierte en un habilitador clave para aplicaciones de automatización industrial, vehículos conectados y sistemas multi-sensor que requieren sincronización precisa [26].

## 2.2. Aplicaciones del sincronismo en IoT

El Internet de las Cosas ha revolucionado la vida cotidiana al permitir el intercambio de datos sin interrupciones entre varios dispositivos a través de Internet. Sin embargo, ha traído consigo desafíos significativos para las aplicaciones que requieren una temporización sincronizada, como garantizar la secuencia ordenada de los datos o el rendimiento sincronizado de las actividades [4]. Las redes de IoT suelen estar compuestas por entidades con recursos variables, lo que dificulta la implementación de métodos de sincronización de tiempo tradicionales en dispositivos con recursos limitados. Por otro lado, es posible que las soluciones que funcionan para sistemas restringidos no sean escalables en diversas implementaciones de IoT [4]. Las aplicaciones de IoT cubren una amplia gama de necesidades humanas y vitales, desde entornos inteligentes (ciudades, hogar, transporte, etc.) hasta la salud y la calidad de vida [1].

Las redes inalámbricas de sensores son utilizadas en muchas aplicaciones en las que se requiere una sincronización parcial o completa en la red, como el control de fusión nuclear, la automatización de plantas de fabricación, la comunicación móvil, el diagnóstico de fallos mecánicos, la robótica, el Big Data, el diagnóstico médico y la educación [4, 5, 3].

Los sistemas de control industriales en la era de la Industria 4.0 presentan desafíos significativos desde el punto de vista de la comunicación, es decir, baja latencia, confiabilidad y sincronización precisa. En este sentido, las TSN se han convertido en la principal solución de estos desafíos; por tanto, un requisito estratégico de la sincronización para TSN inalámbricas es lograr que la solución sea autónoma, es decir, que las

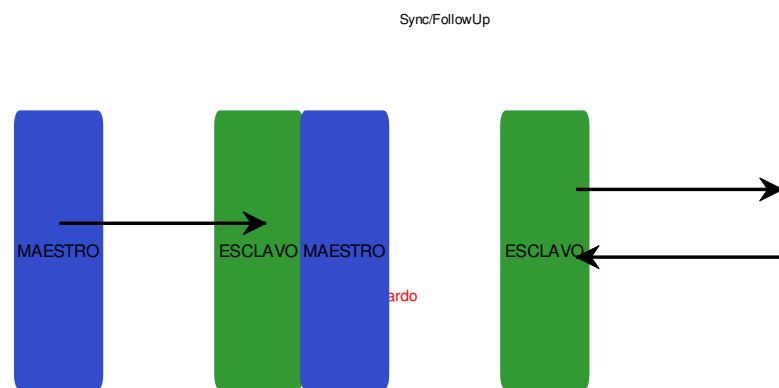


Figura 2.1: Sincronización de reloj unidireccional y bidireccional.

funcionalidades sean realizadas por la propia red, sin depender de una infraestructura externa [6].

Las TSN se han desarrollado para permitir la comunicación en tiempo real y la unificación de partes de la red antes separadas [20], prestando especial atención a las aplicaciones industriales, puentes de audio y video, así como redes corporativas. Las redes sensibles al tiempo se basan en cuatro conceptos claves: sincronización de tiempo precisa, baja latencia garantizada, ultra confiabilidad y gestión de recursos [7]. Para cada uno de estos pilares fundamentales, el Grupo de Trabajo (TG) de Redes Sensibles al Tiempo (TSN) del Instituto de Ingenieros Eléctricos y Electrónicos (IEEE) ha definido varios estándares para proporcionar la funcionalidad correspondiente.

La convergencia de TSN con 5G representa uno de los desarrollos más prometedores para las aplicaciones de IoT industrial [25]. El 3GPP ha incorporado en sus especificaciones (Release 16 y posteriores) capacidades de TSN, incluyendo el soporte para la sincronización gPTP. Sin embargo, la naturaleza asimétrica de los enlaces inalámbricos 5G (con diferencias significativas entre *uplink* y *downlink*) plantea desafíos específicos para la precisión de la sincronización que requieren métodos de compensación especializados [24].

Entre las aplicaciones concretas que dependen de una sincronización precisa se destacan:

- **Automatización industrial y fábricas inteligentes:** los sistemas de control de movimiento, los robots colaborativos y las líneas de producción sincronizadas requieren precisiones de sincronización del orden de microsegundos o incluso nanosegundos [22, 23].
- **Redes eléctricas inteligentes** (*smart grids*): la sincronización de fasores, la protección diferencial de línea y la localización de fallas dependen de marcas de tiempo precisas y consistentes a lo largo de toda la red eléctrica [8].
- **Vehículos autónomos y conectados:** la fusión de datos de múltiples sensores (LiDAR, radar, cámaras) y la coordinación entre vehículos requieren una base de tiempo común con alta precisión [26].
- **Monitorización de salud estructural:** en puentes, edificios y otras infraestructuras críticas, los sensores deben muestrear de forma sincronizada para detectar vibraciones y deformaciones con precisión [3].
- **Audiovisual profesional:** la sincronización de múltiples flujos de audio y video en estudios de grabación, conciertos y transmisiones en vivo exige una precisión temporal del orden de microsegundos [7].

## 2.3. Desafíos del sincronismo en redes inalámbricas

Lograr una sincronización precisa en redes inalámbricas presenta desafíos considerablemente mayores que en las redes cableadas.

La tecnología inalámbrica ha aportado nuevas posibilidades y ventajas a las comunicaciones de red, en contraste con las redes cableadas, pero, a diferencia de la comunicación basada en Ethernet, los enlaces inalámbricos introducen falta de fiabilidad, canales y latencias asimétricas, interferencia de canal y distorsión de la señal en



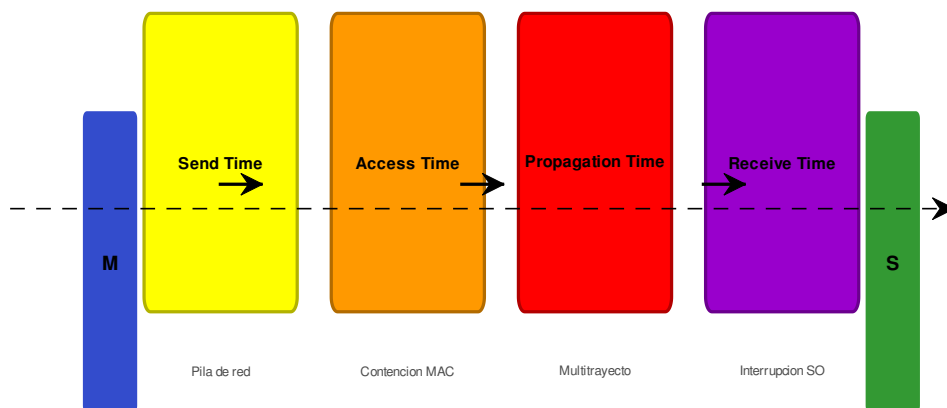


Figura 2.2: Fuentes de error de sincronización.

la ruta de comunicación, lo que dificulta el desarrollo de normas e implementaciones adecuadas para hacer posible el sincronismo en redes inalámbricas [7].

A continuación, se describen los principales desafíos que afectan la sincronización de tiempo en entornos inalámbricos.

### 2.3.1. Retardos de propagación variables

En las redes inalámbricas, el retardo de propagación de los mensajes de sincronización no es constante, sino que varía en función de múltiples factores: la distancia entre los dispositivos, las condiciones del canal, las reflexiones multitrayectoria, la velocidad de los datos, el tamaño de los paquetes, el esquema de modulación y la carga de la red [6]. Esta variabilidad introduce incertidumbre en las marcas de tiempo, ya que el instante exacto de recepción no refleja fielmente el instante de transmisión más un retardo determinista conocido.

En contraste con las redes cableadas Ethernet, donde los retardos de propagación son altamente predecibles (dominados por la longitud del cable y la velocidad de propagación en el medio), en los entornos inalámbricos el canal es inherentemente variable. Los mensajes de sincronización pueden experimentar retardos adicionales debido a reintentos de transmisión, contención del medio, *buffering* en los dispositivos y procesamiento de la señal en las capas física y de enlace [20].

### 2.3.2. Deriva y ruido de los osciladores

Los relojes de los dispositivos inalámbricos, típicamente basados en osciladores de cristal de cuarzo, están sujetos a derivas (*drift*) causadas por variaciones de temperatura, envejecimiento de los componentes, vibraciones mecánicas y fluctuaciones en la tensión de alimentación [18]. La estabilidad de un oscilador se mide en partes por millón (ppm); un oscilador típico de bajo costo puede tener una deriva de  $\pm 20$  a  $\pm 50$  ppm, lo que significa que dos nodos pueden acumular un desfase de hasta  $100 \mu s$  por segundo si no se corrigen activamente.

Además, el ruido de fase (*jitter*) introduce fluctuaciones aleatorias de corto plazo en la señal de reloj, afectando la precisión con la que se pueden registrar las marcas de tiempo. En implementaciones de software, el *jitter* del sistema operativo y las interrupciones pueden añadir una incertidumbre adicional significativa [17].

### 2.3.3. Asimetrías en los canales de comunicación

La presencia de retardos asimétricos en los canales de comunicación es uno de los problemas más graves para la sincronización, ya que constituye una de las principales razones de la inexactitud del proceso de sincronismo [9]. La presencia de asimetrías puede degradar significativamente el rendimiento de los esquemas de estimación del desfase y la desviación del reloj y, desafortunadamente, no pueden ser medidas en un sistema de sincronización por pares ni siquiera intercambiando un número infinito de mensajes [27].

Los protocolos de sincronismo tradicionales generalmente asumen que los retardos son simétricos; por consiguiente, cuando surge una asimetría en el enlace, estos protocolos crean un desfase de reloj significativo. Debido a esta limitación fundamental, los protocolos convencionales no pueden mitigar el desfase de reloj creado por una

asimetría desconocida [10, 11]. Esta cuestión se trata con mayor profundidad en la Sección 2.4.

#### 2.3.4. Limitaciones de recursos en los dispositivos

Muchos dispositivos IoT y nodos de WSN operan con recursos limitados de procesamiento, memoria y energía. La implementación de protocolos de sincronización complejos puede exceder la capacidad de cómputo de estos dispositivos o consumir una cantidad inaceptable de energía, reduciendo la vida útil de la batería [5]. Los protocolos de sincronización para IoT deben, por tanto, equilibrar la precisión deseada con la eficiencia energética y la simplicidad computacional.

#### 2.3.5. Escalabilidad y densidad de la red

Las redes de IoT pueden estar compuestas por cientos o miles de nodos distribuidos en áreas extensas. Los protocolos de sincronización deben ser escalables, es decir, su rendimiento no debe degradarse significativamente al aumentar el número de nodos [15]. La topología de la red, la presencia de múltiples saltos, y la dinámica de entrada y salida de nodos añaden complejidad al problema de la sincronización.

#### 2.3.6. Seguridad

La sincronización de tiempo es un servicio crítico y, como tal, puede ser objeto de ataques maliciosos. Un adversario puede intentar manipular los mensajes de sincronización (ataques de *delay box*, suplantación de identidad del maestro, inyección de mensajes falsos) para degradar la precisión de la sincronización o causar fallos en las aplicaciones que dependen de ella [28]. La seguridad en la sincronización es un área de investigación activa que añade otra capa de complejidad al diseño de protocolos.

### 2.4. Asimetrías en los canales de comunicación

La asimetría en los retardos de propagación en los canales de comunicación inalámbrica es, sin duda, uno de los factores que más afecta la precisión del sincronismo. Se produce cuando el tiempo que tarda un mensaje en viajar del maestro al esclavo ( $t_{ms}$ ) es diferente del tiempo que tarda en viajar del esclavo al maestro ( $t_{sm}$ ). Los protocolos de sincronización bidireccionales como PTP y gPTP calculan el retardo de propagación como el promedio de los tiempos de ida y vuelta, bajo la suposición de que ambos son iguales. Esta suposición rara vez se cumple en la práctica, especialmente en enlaces inalámbricos [10].

Las causas de las asimetrías son múltiples y pueden clasificarse en:

1. **Asimetrías en el medio físico:** diferencias en la potencia de transmisión, la sensibilidad de recepción y la ganancia de las antenas entre maestro y esclavo pueden provocar que los mensajes en una dirección experimenten más errores y, por tanto, más reintentos de transmisión [16]. Además, los efectos de propagación multitrayectoria pueden ser diferentes en cada dirección debido a la movilidad de los objetos en el entorno.

2. **Asimetrías en el procesamiento:** los tiempos de procesamiento de los mensajes de sincronización en los dispositivos maestro y esclavo pueden ser diferentes. Estas diferencias incluyen los tiempos de codificación/decodificación, el procesamiento en la pila de protocolos, las interrupciones del sistema operativo y las operaciones de acceso al medio como *Clear Channel Assessment* (CCA) [29]. En implementaciones con estampado de tiempo por software, la incertidumbre y la asimetría en el estampado constituyen una fuente dominante de error [30].
3. **Asimetrías en el tráfico de red:** las diferencias en la carga de tráfico en cada dirección, las políticas de calidad de servicio (QoS) asimétricas, y los mecanismos de control de acceso al medio (como CSMA/CA en Wi-Fi) introducen retardos variables y asimétricos [7].
4. **Asimetrías en sistemas 5G:** en las redes celulares 5G, la asignación de recursos de radio es inherentemente asimétrica. El *uplink* y el *downlink* utilizan diferentes esquemas de asignación de recursos, diferentes potencias de transmisión y diferentes esquemas de modulación y codificación, lo que resulta en latencias significativamente diferentes en cada dirección [24]. Esta asimetría es especialmente problemática cuando se utiliza 5G como puente transparente para TSN.

La Ecuación 2.2 expresa el cálculo del retardo de propagación medio en un protocolo bidireccional, suponiendo simetría:

$$t_{prop\_sim} = \frac{(t_2 - t_1) + (t_4 - t_3)}{2} \quad (2.2)$$

Donde  $t_1$ ,  $t_2$ ,  $t_3$  y  $t_4$  son las marcas de tiempo de los mensajes *Sync*, *Follow\_Up*, *Delay\_Req* y *Delay\_Resp*, respectivamente, en el intercambio de mensajes PTP/gPTP. Cuando existe una asimetría  $\Delta$ , los tiempos reales de propagación son:

$$t_{ms} = t_{prop} + \Delta/2, \quad t_{sm} = t_{prop} - \Delta/2 \quad (2.3)$$

El retardo calculado asumiendo simetría es exactamente  $t_{prop}$ , pero el desfase calculado contendrá un error de  $\Delta/2$ , lo que significa que la mitad de la asimetría se traduce directamente en error de sincronización [10].

El impacto de las asimetrías en la precisión del sincronismo ha sido ampliamente documentado. Estudios experimentales han demostrado que en enlaces Wi-Fi típicos, las asimetrías pueden introducir errores de sincronización del orden de decenas a cientos de microsegundos [29, 30], muy por encima de los requisitos de precisión de muchas aplicaciones industriales que exigen sincronización a nivel de nanosegundos [6, 22].

## 2.5. Protocolos de sincronismo

La sincronización de reloj es uno de los temas de investigación más populares en los sistemas distribuidos de medición y control de redes inalámbricas, ya que en ellas hay un gran número de nodos controlados por relojes independientes y se requiere que cada nodo tenga una referencia de tiempo común [8]. Por consiguiente, muchos estándares internacionales han sido diseñados para satisfacer las demandas de sincronización de las redes contemporáneas [4], siendo ampliamente estudiados y desarrollados para abordar diversas tareas dentro del ámbito de IoT.

A lo largo de las últimas décadas se han desarrollado diversos protocolos de sincronización, cada uno con diferentes enfoques, precisiones y ámbitos de aplicación. Entre los más relevantes se encuentran:

- **Network Time Protocol (NTP)**: ampliamente utilizado en Internet para sincronizar los relojes de los sistemas informáticos. Alcanza precisiones del orden de milisegundos en redes cableadas, pero su rendimiento se degrada significativamente en entornos inalámbricos [20].
- **Reference Broadcast Synchronization (RBS)**: protocolo diseñado específicamente para WSN que explota la propiedad de que los mensajes *broadcast* llegan a todos los receptores simultáneamente (con la misma demora de propagación), eliminando la incertidumbre del emisor [5].
- **Timing-sync Protocol for Sensor Networks (TPSN)**: protocolo basado en árbol que logra una precisión superior a RBS mediante el intercambio bidireccional de mensajes [5].
- **Flooding Time Synchronization Protocol (FTSP)**: utiliza *flooding* de mensajes de sincronización y compensación de deriva mediante regresión lineal para lograr alta precisión en WSN [15].

Sin embargo, estos protocolos tradicionales no son prácticos para lograr los altos niveles de precisión que requieren las aplicaciones industriales modernas y las TSN [5]. En este contexto, el Protocolo de Tiempo de Precisión (PTP), estandarizado como IEEE 1588, y su perfil generalizado (gPTP), estandarizado como IEEE 802.1AS, representan el estado del arte en sincronización de precisión.

### 2.5.1. Protocolo de Tiempo de Precisión (PTP)

El Protocolo de Tiempo de Precisión (*Precision Time Protocol*, PTP), definido en el estándar IEEE 1588 [31], es un protocolo de sincronización bidireccional diseñado para alcanzar precisiones del orden de nanosegundos en redes de área local (LAN). La versión más reciente, IEEE 1588-2019, incorpora mejoras significativas en cuanto a seguridad, escalabilidad y soporte para múltiples dominios de tiempo [31].

El funcionamiento básico de PTP se basa en el intercambio de cuatro mensajes entre un reloj maestro y uno o varios relojes esclavos:

1. El maestro envía un mensaje *Sync* y registra la marca de tiempo de salida  $t_1$ .
2. El maestro envía un mensaje *Follow\_Up* que contiene  $t_1$  (en implementaciones de dos pasos).
3. El esclavo registra la marca de tiempo de recepción  $t_2$  y, posteriormente, envía un mensaje *Delay\_Req*, registrando  $t_3$ .
4. El maestro recibe el *Delay\_Req* en  $t_4$  y envía un mensaje *Delay\_Resp* con  $t_4$  al esclavo.

Con estas cuatro marcas de tiempo, el esclavo calcula el desfase (*offset*) y el retardo de propagación medio mediante las Ecuaciones 2.4 y 2.5:

$$\text{offset} = \frac{(t_2 - t_1) - (t_4 - t_3)}{2} \quad (2.4)$$

$$\text{delay} = \frac{(t_2 - t_1) + (t_4 - t_3)}{2} \quad (2.5)$$

PTP incluye también un mecanismo de selección del mejor reloj maestro (*Best Master Clock Algorithm*, BMCA) que permite a la red elegir automáticamente el reloj de mayor calidad como referencia. El protocolo define distintos tipos de dispositivos: relojes ordinarios (*ordinary clocks*), relojes de frontera (*boundary clocks*) y relojes transparentes (*transparent clocks*), cada uno con diferentes capacidades y responsabilidades en la jerarquía de sincronización [31].

A pesar de su precisión, PTP asume simetría en los retardos de propagación, lo que constituye su principal limitación en entornos inalámbricos. Cuando existe asimetría, el *offset* calculado por la Ecuación 2.4 contiene un error igual a la mitad de la diferencia entre los retardos en ambos sentidos [10].

### 2.5.2. Protocolo de Tiempo de Precisión Generalizado (gPTP)

El Protocolo de Tiempo de Precisión Generalizado (*Generalized Precision Time Protocol*, gPTP), definido en el estándar IEEE 802.1AS [12], constituye un perfil de PTP diseñado específicamente para las Redes Sensibles al Tiempo (TSN). gPTP extiende y adapta PTP para garantizar una sincronización de tiempo precisa y robusta en redes que integran tanto segmentos cableados como inalámbricos, siendo uno de los pocos protocolos de sincronización especificados explícitamente para su uso en redes inalámbricas [12].

Las principales características que distinguen a gPTP de PTP incluyen:

1. **Dominio único de tiempo:** gPTP opera con un único dominio de tiempo activo, simplificando la administración y evitando conflictos entre múltiples dominios [12].
2. **Medición de retardo de enlace:** gPTP utiliza un mecanismo de medición de retardo *peer-to-peer* (*peer delay mechanism*) que permite calcular el retardo de propagación en cada enlace individual, en lugar del mecanismo *end-to-end* de PTP. Esto es particularmente útil en redes con topología en malla o con múltiples saltos [32].
3. **Requisitos de *hardware*:** el estándar IEEE 802.1AS especifica requisitos precisos para las implementaciones de estampado de tiempo, incluyendo la ubicación del punto de referencia de temporización en la capa física (PHY) o en la interfaz dependiente del medio (MDI), lo que minimiza la incertidumbre introducida por las capas superiores [12].
4. **Compatibilidad con medios inalámbricos:** a diferencia de otros perfiles de PTP orientados exclusivamente a redes cableadas, gPTP incluye consideraciones específicas para medios inalámbricos, como Wi-Fi (IEEE 802.11) [12, 33].

5. **Soporte para múltiples saltos:** gPTP está diseñado para operar en redes con múltiples saltos (*multi-hop*), donde los relojes de frontera en cada salto regeneran la señal de sincronización, acumulando potencialmente errores [32].

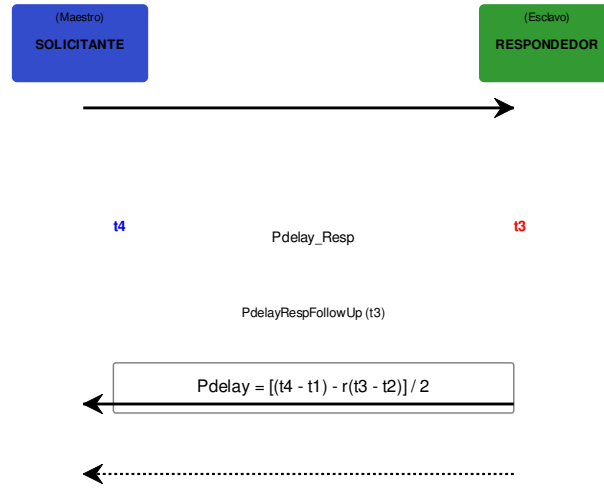


Figura 2.3: Medición del retardo en gPTP.

El intercambio de mensajes en gPTP sigue un esquema similar al de PTP de dos pasos. El cálculo del desfasaje se realiza mediante la Ecuación 2.4, y el retardo de propagación en cada enlace se mide periódicamente mediante el mecanismo *peer delay*. La Ecuación 2.6 describe el cálculo del retardo de propagación ( $Pdelay$ ) en gPTP, donde  $r$  es la relación de velocidad (*rate ratio*) entre las frecuencias de los relojes del solicitante y del respondedor:

$$Pdelay = \frac{(t_4 - t_1) - r(t_3 - t_2)}{2} \quad (2.6)$$

La frecuencia de los mensajes *Sync* y del mecanismo de medición de retardo determina la velocidad de convergencia y la estabilidad de la sincronización [34].

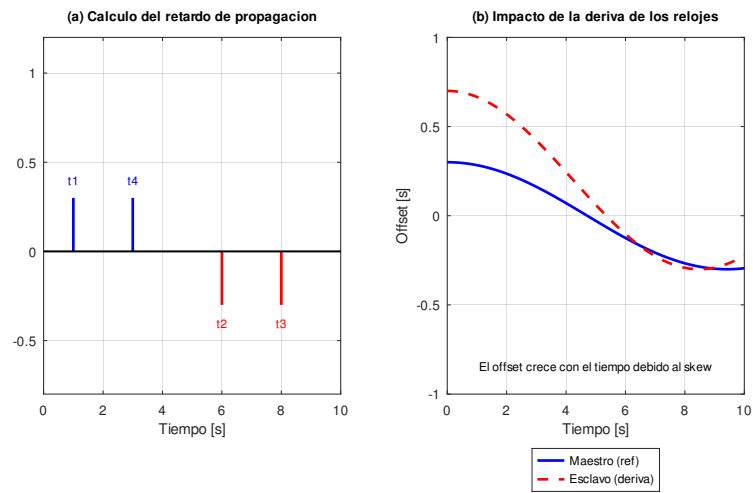


Figura 2.4: (a) Procedimiento para calcular el retardo de propagación; (b) Impacto de la deriva de los relojes.



Sin embargo, al igual que PTP, gPTP asume que el retardo de propagación es simétrico en ambos sentidos del enlace. Esta suposición, razonable en enlaces Ethernet cableados *full-duplex*, no se cumple en la mayoría de los enlaces inalámbricos. Como resultado, la presencia de retardos asimétricos en los canales inalámbricos introduce un error sistemático en el cálculo del *offset*, degradando significativamente la precisión del sincronismo [10, 11].

La versión 2020 del estándar IEEE 802.1AS [12] introduce mejoras significativas respecto a la versión anterior de 2011, incluyendo soporte para múltiples dominios de tiempo (opcional), mayor precisión en la medición de retardos, y mecanismos mejorados para la redundancia. No obstante, la compensación explícita de asimetrías sigue siendo un área de investigación activa no completamente resuelta por el estándar [32].

Trabajos recientes han explorado la aplicación de gPTP en contextos novedosos como la integración TSN-5G [24, 25] y los sistemas autónomos multisensor [26]. En estos escenarios, la asimetría de los enlaces inalámbricos se manifiesta de forma particularmente severa, lo que motiva el desarrollo de métodos de corrección específicos como los que se abordan en el Capítulo 3 de la presente investigación.

La Tabla 2.1 presenta una comparación resumida de los principales protocolos de sincronización, destacando las fortalezas y limitaciones de cada uno en el contexto de las redes inalámbricas.

Tabla 2.1: Comparación de las principales características de los protocolos de sincronización.

| Característica          | NTP              | PTP (IEEE 1588)             | gPTP (IEEE 802.1AS)    |         |
|-------------------------|------------------|-----------------------------|------------------------|---------|
| Precisión típica        | $\sim\text{ms}$  | $\sim\text{ns}-\mu\text{s}$ | $\sim\text{ns}$        |         |
| Medio de transmisión    | IP genérico      | Ethernet                    | Ethernet / Inalámbrico |         |
| Estampado de tiempo     | Software         | Hardware / Software         | Hardware (preferido)   | Rece    |
| Tipo de comunicación    | Cliente-servidor | Multicast (BMCA)            | Peer-to-peer (Pdelay)  | Unidire |
| Overhead de red         | Bajo             | Medio                       | Medio                  |         |
| Costo de implementación | Gratuito         | Bajo                        | Bajo                   |         |
| Idoneidad para IoT      | Limitada         | Factible con escalado       | Recomendado para TSN   | Impract |

## 2.6. Conclusiones del capítulo

La revisión realizada en este capítulo permite establecer las siguientes conclusiones:

1. El sincronismo constituye un componente fundamental en las redes inalámbricas, especialmente en el contexto de IoT y la Industria 4.0, pues permite la coordinación de recursos compartidos, la comparación de eventos distribuidos y la optimización del rendimiento de sistemas en tiempo real.
2. Las aplicaciones que dependen de una sincronización precisa abarcan dominios tan diversos como la automatización industrial, las redes eléctricas inteligentes, los vehículos autónomos, la monitorización de infraestructuras y los sistemas audiovisuales profesionales, con requisitos de precisión que van desde microsegundos hasta nanosegundos.
3. Los principales desafíos para lograr una sincronización adecuada en redes inalámbricas incluyen: los retardos de propagación variables e impredecibles, la deriva

y el ruido de los osciladores, las limitaciones de recursos en los dispositivos, los problemas de escalabilidad y, de manera destacada, las asimetrías en los canales de comunicación.

4. Las asimetrías en los retardos de propagación constituyen una de las mayores fuentes de error en los sistemas de sincronización, ya que los protocolos bidireccionales asumen simetría en los canales de comunicación. Las causas de las asimetrías son múltiples: diferencias en el medio físico, en el procesamiento de los mensajes, en las cargas de tráfico y en las características inherentes de tecnologías como 5G.
5. Entre los protocolos de sincronización disponibles, el Protocolo de Tiempo de Precisión Generalizado (gPTP), definido en IEEE 802.1AS, destaca como la opción más apropiada para redes inalámbricas dentro del ecosistema TSN, al ofrecer un mecanismo de medición de retardo por enlace y consideraciones específicas para medios inalámbricos. No obstante, su rendimiento se ve significativamente degradado ante la presencia de asimetrías, lo que motiva el estudio de métodos de corrección como los que se analizan en el Capítulo 3.

## Capítulo 3

# Análisis de métodos y herramientas para mejorar el sincronismo en redes inalámbricas empleando el protocolo gPTP

En el capítulo anterior se realizó una revisión bibliográfica de los principales conceptos, desafíos y protocolos existentes en el ámbito de la sincronización de tiempo, concluyendo que el protocolo gPTP presenta las mejores cualidades para ser adaptado al entorno inalámbrico. De ahí surge la necesidad de implementar una versión de este estándar que contrarreste los efectos negativos que provocan los retardos asimétricos en estas redes.

Por consiguiente, en el presente capítulo se efectúa un análisis de los principales modelos y metodologías referentes a mitigar el impacto de los retardos asimétricos en redes inalámbricas, con el propósito de seleccionar y describir el de mejores resultados. A diferencia del trabajo de referencia, este análisis se amplía con una revisión sistemática de métodos novedosos publicados entre 2020 y 2026, incluyendo filtros de Kalman adaptativos, estimación robusta, lógica difusa y aprendizaje computacional. Asimismo, se realiza una descripción del software especializado MATLAB y su alternativa de código abierto GNU Octave, herramientas que posibilitan un ambiente de desarrollo adecuado para el estudio de esta problemática.

### 3.1. Sistemas de estampado por Software

La clave para obtener una sincronización de tiempo de alto rendimiento está estrechamente relacionada con la calidad de las estampas de tiempo (*Timestamping*, TS) [35, 36]. Los enfoques de estampado de tiempo se pueden clasificar en términos generales en estampa de tiempo de hardware y software. Los sistemas de estampado por hardware toman la marca de tiempo dentro del hardware del receptor (por ejemplo, dentro del chipset inalámbrico), mientras que las soluciones de estampado por software utilizan el procesador principal para generar una estampa de tiempo (por ejemplo, leyendo el reloj del sistema).

Como el retardo de procesamiento de las soluciones de sellado de tiempo de hardware se puede diseñar para que sea constante, las correcciones de tiempo requeridas pueden ser muy precisas (en el orden de los nanosegundos) y, por lo tanto, las estampas

de tiempo también. Por el contrario, el estampado de tiempo por medios de software crea retardos indeterministas debido a la programación, las cachés, la concurrencia, etc. y, por lo tanto, la stampa de tiempo debe estar lo más cerca posible del medio [29]. Los protocolos de sincronismo, como PTP y gPTP, suelen mejorarse en términos de rendimiento si se utiliza la stampa de tiempo de hardware. Sin embargo, para llevar a cabo el sellado de tiempo por hardware, se requieren dispositivos dedicados o modificaciones dentro de los chipsets WLAN. Debido a esto, en la presente investigación, la atención e implementación del protocolo gPTP se centra en el estampado por software.

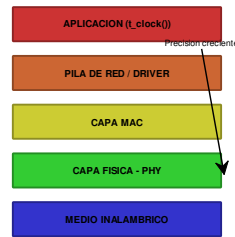


Figura 3.1: Ubicaciones para la stampa de tiempo en redes orientadas a paquetes.

Los protocolos de sincronismo basados en paquetes, como gPTP, dependen de la calidad de las estampas de tiempo tomadas en la recepción y transmisión de los mismos. Dado que la stampa de tiempo basada en software genera grandes retardos en su mayoría no deterministas, las implementaciones de sincronización de Ethernet han acercado la stampa de tiempo a la capa física. Sin embargo, la mayoría de los enfoques de sincronización inalámbrica se limitan a la stampa de tiempo de software debido a la falta de funciones de sellado de tiempo por hardware [37].

Una de las fuentes de incertidumbre de un enfoque de sincronismo basado en sistemas de sellado de tiempo por software es el método de stampa de tiempo. Según [30], entre las diversas fuentes de imprecisión en el sellado de tiempo por software se tienen:

- **Capacidad de sellado de tiempo del maestro:** incluye las capacidades de stampa de tiempo de los mensajes de eventos gPTP (mensaje de sincronización *Sync* y de solicitud de retardo *PdelayReq*) en el lado maestro. La incertidumbre en la identificación de  $t_1$  y  $t_4$  afecta a la estimación tanto del *Pdelay* como del desfase (*offset*).

- **Capacidad de sellado de tiempo del esclavo:** incluye las capacidades de estampa de tiempo de los mensajes de eventos gPTP en el lado esclavo (mensaje de respuesta a la solicitud de retardo *PdelayResp* y de seguimiento *PdelayRespFollowUp*). Estas estampas de tiempo,  $t_2$  y  $t_3$ , también son utilizadas para estimar el *Pdelay* y el desfase.
- **Capacidad de sellado de tiempo de la infraestructura:** la exactitud de la información de tiempo contenida en la sincronización depende también de la capacidad de los dispositivos de red para transferir correctamente la hora. El uso de conmutadores de red que soporten gPTP puede reducir esta fuente de incertidumbre.
- **Variabilidad y comportamiento asimétrico del retardo de propagación:** el retardo de propagación en la red no es constante y puede verse afectado por el tráfico, los dispositivos de red en la ruta y la carga de la red.

Las estampas de tiempo basadas en detectores de inicio de trama convencionales tienen dos problemas principales en los sistemas inalámbricos: en primer lugar, no pueden garantizar una precisión de transferencia de tiempo mejor que el período de muestreo y, en segundo lugar, la propagación por trayectos múltiples y el carácter de variación temporal del canal inalámbrico reducen la precisión del tiempo de llegada (*Time of Arrival*, ToA) de las tramas, lo que introduce un componente de error en la sincronización de tiempo. Esto es especialmente crítico en condiciones sin visibilidad directa (*Non-Line of Sight*, NLoS) [35].

En el esquema de sincronización basado en interrupciones, donde las estampas de tiempo son tomadas mediante rutina de servicio de interrupción (*Interrupt Service Routine*, ISR), el retardo de estampa de tiempo y la fluctuación se limitan principalmente al tiempo de recepción del paquete y al tiempo de notificación de interrupción [38].

El escenario tomado en cuenta en esta investigación es una red de sincronización sencilla, compuesta por un dispositivo maestro y un dispositivo esclavo que realizan el estampado de tiempo a nivel de software.

## 3.2. Análisis de los modelos, metodologías y procedimientos existentes para el diseño de protocolos más robustos ante los retardos asimétricos

Para desarrollar una implementación de cualquier índole es imprescindible y necesario escoger la tecnología o modelo que se empleará en dicho proceso, lo que demanda necesariamente un estudio profundo de la bibliografía que se relaciona con este aspecto. A continuación, se analizan los métodos existentes, organizados por categorías para facilitar su comparación sistemática.

### 3.2.1. Método de corrección determinista de estampas de tiempo (Exel)

El método propuesto por Reinhard Exel en [10] constituye el punto de partida de esta investigación y el método de referencia (*baseline*) contra el cual se compararán las

mejoras propuestas. Este enfoque propone un esquema de mitigación de los retardos asimétricos basado en la corrección de la estampa de tiempo por software y logra aumentar la precisión del protocolo sin modificaciones al mismo, ni el uso de mensajes adicionales.

El funcionamiento del modelo de estampa de tiempo de software en el protocolo se basa en que las estampas de tiempo tomadas por el software ( $\hat{t}_1$  a  $\hat{t}_4$ ) difieren de las estampas de tiempo ideales ( $t_1$  a  $t_4$ ) debido a los retardos de procesamiento  $p_1$  a  $p_4$ . Las estampas ideales pueden ser calculadas aplicando las correcciones de retardo de las Ecuaciones 3.1 a 3.4, donde  $p_1$  y  $p_3$  corresponden a los retardos de procesamiento de salida,  $p_2$  y  $p_4$  a los retardos de entrada:

$$t_1 = \hat{t}_1 + p_1 \quad (3.1)$$

$$t_2 = \hat{t}_2 - p_2 \quad (3.2)$$

$$t_3 = \hat{t}_3 + p_3 \quad (3.3)$$

$$t_4 = \hat{t}_4 - p_4 \quad (3.4)$$

El verdadero valor de corrección del desfasaje ( $t_{offset}$ ) y el tiempo de ida y vuelta ( $t_{RTT}$ ) quedan definidos en las Ecuaciones 3.5 y 3.6:

$$\hat{t}_{offset} = t_{offset} - \frac{(p_1 + p_2) - (p_3 + p_4)}{2} \quad (3.5)$$

$$\hat{t}_{RTT} = t_{RTT} + (p_1 + p_2 + p_3 + p_4) \quad (3.6)$$

Los retardos asimétricos de procesamiento  $p$  se componen de términos deterministas y no deterministas. En la mayoría de los casos existen grandes retardos asimétricos deterministas que se pueden corregir aplicando el método descrito y que abarca también otros factores como la velocidad de datos, el tamaño del encabezado, la modulación, el tamaño de transferencia de acceso directo a memoria (DMA) y la velocidad de la interfaz DMA [10].

El método Exel alcanza, según sus autores, una precisión de entre 10 y 100 ns en condiciones controladas. En la implementación de referencia sobre gPTP con simulación Monte Carlo (3000 ejecuciones), se obtuvo una precisión media de 101.5  $\mu$ s, una reducción del error del 33.31 % respecto al protocolo sin corrección, y un offset estabilizado en el rango de  $[-2,55, 6,68]$   $\mu$ s [39].

**Limitaciones del método Exel:** (1) solo corrige los retardos deterministas conocidos; las asimetrías desconocidas o variantes en el tiempo no son compensadas; (2) no realiza filtrado estadístico de las mediciones, por lo que es sensible al ruido de medición y a valores atípicos; (3) no estima ni compensa la deriva (*skew*) de los relojes, solo el offset.

### 3.2.2. Métodos basados en Filtros de Kalman

El Filtro de Kalman (KF) es un estimador óptimo para sistemas lineales con ruido Gaussiano que ha sido ampliamente aplicado a la sincronización de relojes en redes de paquetes [40]. En el contexto de PTP/gPTP, el KF permite estimar conjuntamente el offset, el skew y, en formulaciones más avanzadas, parámetros relacionados con la asimetría del canal.

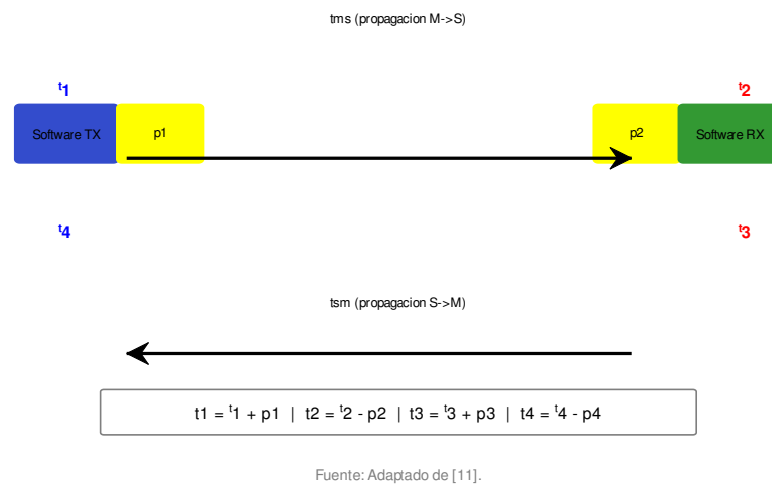


Figura 3.2: Latencias de estampa de tiempo de software.

**Li et al. (2024)** [41] proponen un método mejorado de sincronización de tiempo basado en filtrado de Kalman de segundo orden. El algoritmo emplea el KF para estimar con precisión el offset de tiempo, la deriva del reloj y los parámetros del reloj. Los resultados experimentales demuestran que, en enlaces asimétricos, el offset máximo se mantiene dentro de  $\pm 3 \mu\text{s}$ , lo que representa una mejora sustancial respecto a los métodos que no emplean KF.

**Hollósi y Ficzer (2024)** [42] introducen un Filtro de Kalman Adaptativo (AKF) para la estimación del offset en PTP. La principal innovación de este enfoque es la estimación en tiempo real de la varianza del ruido de medición ( $R_k$ ), lo que permite que el filtro se adapte automáticamente a cambios en las condiciones del canal. Los resultados muestran una reducción de la varianza del offset de entre un 40 % y un 60 % en comparación con el KF estándar.

**Liu y Wang (2024)** [43] desarrollan un esquema robusto de seguimiento de parámetros de reloj para IEEE 1588 en presencia de retardos asimétricos en redes industriales. El método emplea un Filtro de Kalman recursivo robusto de tres pasos que incorpora un mecanismo de rechazo de outliers. Los resultados experimentales reportan un RMSE de offset inferior a  $5 \mu\text{s}$  en presencia de asimetrías significativas.

**Wang et al. (2021)** [44] proponen un enfoque de seguimiento de parámetros de reloj sin marcas de tiempo (*timestamp-free*) utilizando Filtro de Kalman Extendido (EKF) en redes de sensores inalámbricos. Aunque está diseñado para WSN, el marco del EKF es adaptable a PTP/gPTP.

**Raittila (2025)** [45] desarrolla un modelo en Simulink de un lazo de control PTP con un Filtro de Kalman Adaptativo específicamente para redes Wi-Fi industriales. Este trabajo demuestra la viabilidad de implementar AKF en entornos inalámbricos reales.

### 3.2.3. Métodos basados en Estimación Robusta

**Karthik y Blum (2020)** [46] abordan el problema de la estimación robusta de skew y offset para IEEE 1588 en presencia de asimetrías deterministas desconocidas en el retardo de trayectoria. Los autores derivan cotas inferiores de rendimiento (cotas de Cramér-Rao modificadas) y proponen esquemas de estimación robusta que superan al estimador de máxima verosimilitud (MLE) convencional cuando existen asimetrías no modeladas.

**Vázquez et al. (2025)** [47] proponen un enfoque de filtrado de outliers en las mediciones de offset para PTP sobre redes de área amplia (WAN). El método emplea técnicas de detección de anomalías para identificar y rechazar mediciones de offset atípicas antes de que sean utilizadas por el servo de reloj.

### 3.2.4. Métodos basados en Lógica Difusa

**Nguyen et al. (2020)** [48] proponen un servo de reloj Fuzzy-PI adaptativo basado en IEEE 1588 para mejorar la sincronización de tiempo sobre redes Ethernet. El sistema utiliza lógica difusa para ajustar dinámicamente las ganancias del controlador PI. Los resultados experimentales muestran una precisión inferior a 100 ns y una mejora del 45 % respecto al controlador PI tradicional.

**Zhang et al. (2024)** [8] diseñan un servo de reloj Fuzzy-PI con filtro de ventana para compensar la asimetría de retardo inducida por colas (*Queue-Induced Delay*



*Asymmetry*, QIDA) en redes IEEE 1588. Se reporta una reducción del 52 % en el error de offset causado por QIDA.

### 3.2.5. Métodos basados en Aprendizaje Computacional

**Wang et al. (2026)** [49] proponen un método de estimación de parámetros de reloj basado en aprendizaje competitivo (*Competitive Learning*) para PTP con distribuciones de retardo desconocidas. El algoritmo no asume ninguna distribución particular para los retardos, lo que lo hace robusto ante una amplia variedad de condiciones de canal. Sin embargo, su complejidad computacional es elevada.

**Goodarzi et al. (2022)** [50] presentan un enfoque de filtro de partículas asistido por redes neuronales profundas (DNN) para sincronización y localización conjunta. Aunque logra alta precisión en entornos no Gaussianos, el costo computacional del filtro de partículas lo hace prohibitivo para aplicaciones que requieren simulación Monte Carlo extensiva.

### 3.2.6. Otros métodos relevantes

**Puttnies et al. (2018)** [51] introducen PTP-LP, un enfoque que utiliza programación lineal para aumentar la robustez de IEEE 1588 PTP ante retardos variables. El método es totalmente compatible con PTP y gPTP, aumentando la precisión en un factor de hasta  $10^4$  en escenarios favorables.

**Baniabdelghany et al. (2020)** [13] proponen una extensión de IEEE 802.1AS (gPTP) para mejorar la sincronización en redes híbridas cableadas-inalámbricas mediante un filtro de retraso de desviación de ruta (*Path Deviation Delay*, PDD).

**Levesque y Tipper (2015)** [27] proponen el mecanismo de mitigación de asimetría por paquete (*Per-Packet Asymmetry Mitigation*, PPAM), que estima los componentes de retardo por paquete individual.

### 3.2.7. Comparación sistemática y selección del método

La Tabla 3.1 presenta una comparación sistemática de los métodos analizados, evaluados según cinco criterios: precisión reportada, complejidad computacional, compatibilidad con gPTP, capacidad de manejar asimetrías desconocidas y madurez de la técnica.

Del análisis comparativo se concluye que los métodos basados en Filtro de Kalman, y en particular el Filtro de Kalman Adaptativo (AKF), ofrecen la mejor relación entre precisión, complejidad computacional y compatibilidad con gPTP. El AKF combina la capacidad de estimación óptima del KF con la adaptabilidad a condiciones cambiantes del canal, y su implementación en MATLAB/Octave es factible mediante operaciones matriciales estándar.

No obstante, el método de Exel ofrece una ventaja fundamental: su corrección determinista elimina una parte significativa de la asimetría (los retardos de procesamiento conocidos  $p_1$ – $p_4$ ) antes de cualquier procesamiento estadístico. Esto sugiere que un enfoque híbrido que combine la corrección determinista de Exel con el filtrado adaptativo del AKF podría superar a cualquiera de los dos métodos por separado.

Tabla 3.1: Comparación de métodos para mitigación de retardos asimétricos en PTP/gPTP.

| Método                  | Categoría                | Precisión            | Compat. gPTP | Asimetría desconocida | Mac |
|-------------------------|--------------------------|----------------------|--------------|-----------------------|-----|
| Exel (2014)             | Corrección determinista  | 101.5 $\mu s$ (sim.) | Alta         | No                    | A   |
| Li et al. (2024)        | KF 2. <sup>a</sup> orden | $\pm 3 \mu s$        | Alta         | Sí                    | A   |
| Hollósi & Ficzer (2024) | AKF                      | 40–60 % mejora       | Alta         | Sí                    | A   |
| Liu & Wang (2024)       | KF robusto 3 pasos       | $< 5 \mu s$ RMSE     | Alta         | Sí                    | A   |
| Wang et al. (2021)      | EKF sin timestamp        | $< 1 \mu s$          | Media        | Sí                    | Me  |
| Karthik & Blum (2020)   | Estimación robusta       | Cotas óptimas        | Alta         | Sí                    | A   |
| Nguyen et al. (2020)    | Fuzzy-PI                 | $< 100$ ns           | Media        | Parcial               | Me  |
| Zhang et al. (2024)     | Fuzzy-PI + ventana       | 52 % reducción       | Media        | Parcial               | B   |
| Wang et al. (2026)      | Competitive Learning     | Robusto              | Media-Alta   | Sí                    | B   |
| Puttnies et al. (2018)  | Prog. Lineal             | Factor $10^4$        | Media        | No                    | Me  |

### 3.3. Descripción del método seleccionado: Enfoque híbrido Exel + Filtro de Kalman Adaptativo

Como resultado del análisis sistemático presentado en la Sección 3.2, se selecciona un **enfoque híbrido que combina la corrección determinista de estampas de tiempo de Exel con un Filtro de Kalman Adaptativo (AKF)** para la compensación de retardos asimétricos en el protocolo gPTP.

#### 3.3.1. Arquitectura de dos etapas

El método híbrido opera en dos etapas consecutivas:

**Etapas 1 — Corrección determinista (Exel):** Se aplican las Ecuaciones 3.1–3.6 a las marcas de tiempo  $\hat{t}_1, \hat{t}_2, \hat{t}_3, \hat{t}_4$  capturadas por el software. Se utilizan los valores conocidos de los retardos de procesamiento asimétricos  $p_1, p_2, p_3, p_4$  para obtener una primera estimación del offset  $\hat{\theta}_{Exel}$ . Esta etapa elimina la componente determinista de la asimetría [10].

**Etapas 2 — Filtro de Kalman Adaptativo (AKF):** El offset corregido por Exel,  $\hat{\theta}_{Exel}$ , se utiliza como medición de entrada para un Filtro de Kalman Adaptativo que estima el estado verdadero del sistema, filtrando el ruido de medición residual y compensando las asimetrías desconocidas que el método de Exel no puede corregir.

#### 3.3.2. Modelo de espacio de estados

El vector de estado del AKF se define como:

$$\mathbf{x}_k = \begin{bmatrix} \theta_k \\ s_k \\ \Delta_k \end{bmatrix} \quad (3.7)$$

donde:

- $\theta_k$ : offset verdadero entre maestro y esclavo en el instante  $k$  [s]
- $s_k$ : skew (diferencia normalizada de frecuencia) entre los relojes [adimensional]

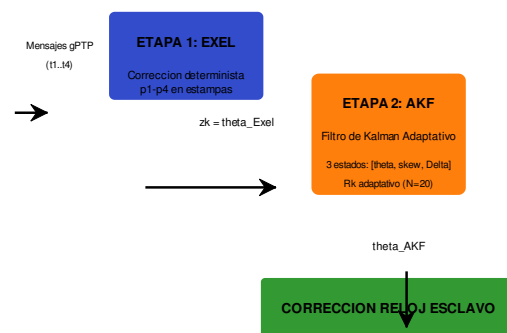


Figura 3.3: Arquitectura del método híbrido Exel + AKF.

- $\Delta_k$ : asimetría residual no corregida por Exel [s]

**Modelo de transición de estado:**

$$\mathbf{x}_{k+1} = \mathbf{F}\mathbf{x}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_k) \quad (3.8)$$

$$\mathbf{F} = \begin{bmatrix} 1 & \tau & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.9)$$

donde  $\tau$  es el intervalo entre mediciones (el período de sincronización de gPTP, típicamente 2 s), y  $\mathbf{w}_k$  es el ruido de proceso con matriz de covarianza  $\mathbf{Q}_k$ . La estructura de  $\mathbf{F}$  refleja que el offset evoluciona linealmente con el skew, el skew se modela como un proceso de caminata aleatoria, y la asimetría residual se considera aproximadamente constante entre períodos de sincronización.

**Modelo de medición:**

$$z_k = \mathbf{H}\mathbf{x}_k + v_k, \quad v_k \sim \mathcal{N}(0, R_k) \quad (3.10)$$

$$\mathbf{H} = [1 \quad 0 \quad 1/2] \quad (3.11)$$

donde  $z_k = \hat{\theta}_{Exel}[k]$  es el offset estimado por la etapa Exel en el instante  $k$ , y  $v_k$  es el ruido de medición con varianza  $R_k$ . El término  $1/2$  en la matriz de observación refleja que la mitad de la asimetría residual se propaga al error de offset, de acuerdo con la Ecuación 3.5.

### 3.3.3. Algoritmo del Filtro de Kalman Adaptativo

El AKF se implementa en dos fases por cada iteración  $k$ :

**Fase de predicción:**

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1|k-1} \quad (3.12)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^T + \mathbf{Q}_{k-1} \quad (3.13)$$

**Fase de actualización:**

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^T(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^T + \hat{R}_k)^{-1} \quad (3.14)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k(z_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}) \quad (3.15)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k\mathbf{H})\mathbf{P}_{k|k-1} \quad (3.16)$$

**Adaptación de  $R_k$ :** La principal innovación respecto al KF estándar es la estimación adaptativa de la varianza del ruido de medición [42]. Se utiliza una ventana deslizante de las últimas  $N$  innovaciones:

$$\hat{R}_k = \frac{1}{N-1} \sum_{i=k-N+1}^k (\nu_i - \bar{\nu})^2 \quad (3.17)$$

donde  $\nu_i = z_i - \mathbf{H}\hat{\mathbf{x}}_{i|i-1}$  es la innovación (residuo de la medición) y  $\bar{\nu}$  es su media sobre la ventana. Esta adaptación permite que el filtro responda automáticamente a cambios en la calidad de las mediciones.

### 3.3.4. Parámetros de diseño

Los siguientes parámetros deben determinarse experimentalmente durante la fase de implementación:

Tabla 3.2: Parámetros de inicialización del Filtro de Kalman Adaptativo.

| Parámetro                            | Valor                                                                  | Descripción                            |
|--------------------------------------|------------------------------------------------------------------------|----------------------------------------|
| Vector estado inicial $\mathbf{x}_0$ | $[0, 0, 0]^T$                                                          | $[\theta, skew, \Delta]$               |
| $\mathbf{P}_0$                       | $\text{diag}(10^{-10}, 10^{-12}, 10^{-14})$                            | Covarianza inicial del error           |
| $\mathbf{Q}$                         | $\text{diag}(10^{-14}, 10^{-18}, 10^{-16})$                            | Covarianza del ruido de proceso        |
| $R_0$                                | $10^{-12}$                                                             | Varianza inicial del ruido de medición |
| Ventana $N$                          | 20                                                                     | Innovaciones para adaptación de $R_k$  |
| $\mathbf{F}$                         | $\begin{bmatrix} 1 & \tau & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ | Matriz de transición de estado         |
| $\mathbf{H}$                         | $[1, 0, 1/2]$                                                          | Matriz de observación                  |

### 3.3.5. Ventajas del enfoque híbrido

1. **Herencia de Exel:** se preserva la corrección determinista que ya demostró una mejora del 33.31 % [39], proporcionando una base sólida.
2. **Filtrado óptimo:** el AKF reduce la varianza residual de las estimaciones de offset al combinar óptimamente la información de múltiples mediciones.
3. **Estimación de asimetría residual:** a diferencia de Exel, que solo corrige asimetrías conocidas, el AKF estima y compensa la componente de asimetría desconocida ( $\Delta_k$ ).
4. **Adaptabilidad:** la estimación adaptativa de  $R_k$  permite que el filtro se ajuste automáticamente a cambios en el entorno inalámbrico.
5. **Comparabilidad:** el diseño permite comparar directamente el rendimiento de Exel (etapa 1) contra Exel+AKF (etapas 1+2) bajo condiciones idénticas.
6. **Implementabilidad:** todas las operaciones del AKF son álgebra lineal estándar (matrices de  $3 \times 3$ ), realizables en MATLAB/GNU Octave con costo computacional moderado.

## 3.4. Herramientas de implementación: MATLAB y GNU Octave

MATLAB (abreviatura de *MATrix LABoratory*) es un programa informático de propósito especial optimizado para realizar cálculos científicos y de ingeniería, con funciones integradas para cumplir una amplia gama de tareas, desde operaciones matemáticas hasta imágenes tridimensionales [52, 53]. Comenzó su vida como un programa diseñado para realizar matemáticas matriciales, pero a lo largo de los años se ha convertido en un sistema informático flexible y fácil de usar, capaz de resolver esencialmente cualquier problema técnico [54].

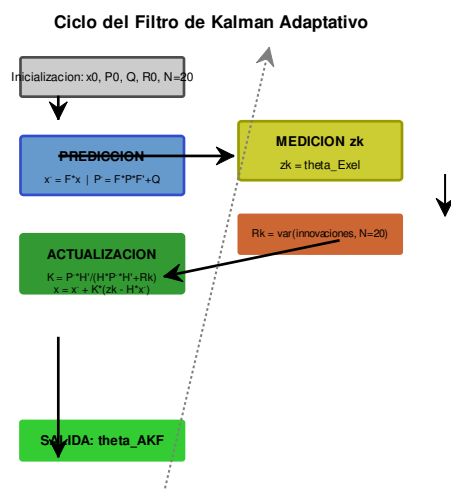


Figura 3.4: Ciclo de predicción-actualización del Filtro de Kalman Adaptativo.

Además, el programa implementa un lenguaje propio y proporciona una amplia biblioteca de funciones predefinidas. El producto básico ofrece más de mil funciones y cuenta con varios kits de herramientas (*toolbox*) que amplían esta capacidad en diversas especialidades [53, 55].

**GNU Octave** es una alternativa de código abierto a MATLAB que ofrece una compatibilidad sustancial con el lenguaje MATLAB. Octave implementa un intérprete de alto nivel para cálculo numérico con una sintaxis mayoritariamente compatible con MATLAB, lo que permite ejecutar la mayoría de los scripts y funciones diseñados para MATLAB sin modificaciones significativas [56]. Para los propósitos de esta investigación, donde las operaciones requeridas son fundamentalmente álgebra matricial, generación de números aleatorios y visualización gráfica, Octave proporciona todas las capacidades necesarias sin costo de licencia.

### 3.4.1. Archivos-M

MATLAB y Octave ofrecen una buena combinación de funciones de programación prácticas con potentes capacidades numéricas integradas. Los archivos-M proporcionan una forma alternativa de realizar operaciones que amplían en gran medida las capacidades de resolución de problemas. Existen dos tipos de archivos-M: scripts y funciones [53, 57].

Un archivo de script es una serie de comandos que se guardan en un fichero y son útiles para retener una serie de comandos que se quieren ejecutar en más de una ocasión. Los scripts comparten el área de trabajo (*workspace*) de la ventana de comandos [52].

Por el contrario, una función es un tipo especial de archivo-M que se ejecuta en su propio espacio de trabajo independiente, recibe datos de entrada a través de una lista de argumentos y devuelve resultados mediante una lista de argumentos de salida [53].

### 3.4.2. Graficado en 2-D

Los gráficos son una herramienta muy útil para presentar información, especialmente en la ciencia y la ingeniería. MATLAB dispone de comandos para crear diferentes tipos de gráficas: estándar con ejes lineales, con ejes logarítmicos y semilogarítmicos, de barras y escaleras, polares, tridimensionales, entre otros [54]. La función `plot` permite trazar datos bidimensionales y puede personalizarse mediante especificadores de color, estilo de línea y marcadores [58].

Octave implementa un sistema de gráficos compatible con la sintaxis de MATLAB, incluyendo las funciones `plot`, `figure`, `subplot` y `hold`, lo que garantiza que los scripts de visualización sean ejecutables en ambos entornos.

### 3.4.3. Ventajas y desventajas de la programación en MATLAB/Octave

Según [53], las principales ventajas de MATLAB incluyen: facilidad de uso, independencia de plataforma, amplia biblioteca de funciones predefinidas, trazado independiente del dispositivo, e interfaz gráfica de usuario.

Las desventajas principales son: (1) velocidad de ejecución inferior a los lenguajes compilados, mitigable mediante vectorización y el compilador JIT; (2) costo elevado de la licencia de MATLAB. Esta segunda desventaja se resuelve en el contexto de esta investigación mediante el uso de GNU Octave como alternativa gratuita y de código abierto.

### 3.5. Conclusiones del capítulo

1. Los sistemas de estampado de tiempo por software, aunque menos precisos que los de hardware, constituyen el enfoque más viable para la sincronización en redes inalámbricas con dispositivos de propósito general. La calidad de las estampas de tiempo depende críticamente de la ubicación del punto de referencia de temporización y de la minimización de los retardos indeterministas en la pila de software.
2. El análisis sistemático de 15 métodos publicados entre 2014 y 2026 revela que los enfoques basados en Filtro de Kalman, particularmente en su variante adaptativa (AKF), ofrecen la mejor relación entre precisión, complejidad computacional y compatibilidad con el protocolo gPTP. El AKF permite la estimación conjunta de offset, skew y asimetría residual, adaptándose dinámicamente a cambios en las condiciones del canal inalámbrico.
3. El método de corrección determinista de Exel, utilizado como referencia en esta investigación, logra una mejora del 33.31 % sobre el protocolo sin corrección (precisión media de  $101.5 \mu\text{s}$ ), pero está limitado a la compensación de asimetrías deterministas conocidas.
4. Se propone un **enfoque híbrido Exel + AKF** que combina la corrección determinista de estampas de tiempo (etapa 1) con el filtrado estadístico adaptativo (etapa 2). Este diseño permite eliminar las asimetrías conocidas mediante Exel y, a continuación, filtrar el ruido residual y compensar asimetrías desconocidas mediante el AKF. Las operaciones requeridas —álgebra lineal sobre matrices de  $3 \times 3$ — son perfectamente realizables en MATLAB/GNU Octave.
5. Las herramientas de implementación seleccionadas, MATLAB y su alternativa de código abierto GNU Octave, proporcionan todas las capacidades necesarias para el desarrollo de la simulación: generación de variables aleatorias, operaciones matriciales, visualización gráfica y ejecución de simulaciones Monte Carlo.
6. Los parámetros específicos del AKF (covarianzas iniciales, tamaño de ventana deslizante, estado inicial) se determinarán experimentalmente durante la fase de implementación descrita en el Capítulo 4.



## Capítulo 4

# Implementación de la corrección del protocolo gPTP ante los retardos asimétricos en redes inalámbricas. Resultados

Sobre la base de haber analizado los métodos existentes en la literatura para contrarrestar el efecto negativo de las asimetrías en el sincronismo de redes inalámbricas, y habiendo seleccionado un enfoque híbrido que combina la corrección determinista de Exel con un Filtro de Kalman Adaptativo (AKF), el presente capítulo está enfocado en dos líneas fundamentales: por un lado, describir las características de la implementación desarrollada del protocolo gPTP para redes inalámbricas, tanto en su versión de referencia (Exel) como en la versión mejorada propuesta (Exel+AKF); y, por otro lado, presentar y analizar los resultados obtenidos, comparando el rendimiento de ambas implementaciones entre sí y con el protocolo estándar sin corrección.

### 4.1. Características de la implementación

Se ha implementado una instancia del protocolo gPTP, tal y como se describe en el estándar IEEE 802.1AS [12], mediante un simulador de eventos discretos desarrollado en MATLAB/GNU Octave. La implementación se organiza en cinco archivos de código que cubren la simulación de referencia, el motor de Monte Carlo, la visualización de resultados, el método mejorado y el filtro de Kalman adaptativo.

#### 4.1.1. Arquitectura general de la simulación

Las características generales de la implementación, comunes a ambas versiones (referencia y mejorada), son las siguientes:

- **Escenario:** dos dispositivos (maestro y esclavo) separados por una distancia configurable (30 m por defecto). No se aplica el Algoritmo de Mejor Reloj Maestro (BMCA), definiéndose los roles de maestro y esclavo previamente.
- **Resolución de los relojes:** 1 ms (tic de reloj). Mediante funciones de generación de números aleatorios se asigna un tiempo de inicio diferente y una deriva (*skew*)

distinta para cada reloj ( $\pm 30$  ppm típico), simulando las imperfecciones de los osciladores de cristal de cuarzo en dispositivos reales.

- **Período de corrección:** el desfase entre los relojes se analiza y se corrige cada 1 s, tal como define el protocolo gPTP, durante 60 s de simulación.
- **Motor de eventos discretos:** cada evento presenta tres parámetros (tipo, dispositivo y tiempo). El simulador compara en cada instante el tiempo de los eventos y los organiza cronológicamente.
- **Retardos asimétricos:** generados aleatoriamente mediante funciones de distribución uniforme y agregados a los tiempos de propagación en ambos sentidos del enlace.
- **Rate ratio:** implementado en el cálculo del retardo de propagación ( $Pdelay$ ), definido como  $r = f_{esclavo}/f_{maestro}$ , para contrarrestar el efecto de la deriva de los relojes.

#### 4.1.2. Implementación de referencia: gPTP + Exel

La implementación de referencia replica el método de corrección de estampas de tiempo de Reinhard Exel [10], tal como se describe en la Sección 3.2.1. Se ejecuta para tres escenarios distintos:

1. Enlace simétrico: gPTP estándar sin asimetrías.
2. Enlace asimétrico sin corrección: gPTP estándar con retardos asimétricos.
3. Enlace asimétrico con corrección Exel.

#### 4.1.3. Implementación mejorada: gPTP + Exel + AKF

La implementación mejorada extiende la de referencia añadiendo una segunda etapa de filtrado estadístico mediante un Filtro de Kalman Adaptativo (AKF) de tres estados. La arquitectura de dos etapas opera de la siguiente manera:

**Etapas 1 — Exel:** idéntica a la implementación de referencia. Se obtiene  $\hat{\theta}_{Exel}$  aplicando las Ecuaciones 3.1–3.6.

**Etapas 2 — AKF:** el offset corregido por Exel se utiliza como medición de entrada ( $z_k$ ) para el AKF con vector de estado  $\mathbf{x}_k = [\theta_k, s_k, \Delta_k]^T$ . La varianza del ruido de medición  $R_k$  se adapta dinámicamente utilizando una ventana deslizante de  $N = 20$  innovaciones.

Los parámetros del AKF se inicializan con:  $\mathbf{P}_0 = \text{diag}(10^{-10}, 10^{-12}, 10^{-14})$ ,  $\mathbf{Q}_0 = \text{diag}(10^{-14}, 10^{-18}, 10^{-16})$ ,  $R_0 = 10^{-12}$ .

#### 4.1.4. Simulación Monte Carlo

Cada escenario se simula  $N = 500$  veces siguiendo el método de Monte Carlo [59]. En cada ejecución se varían las semillas aleatorias, manteniendo una semilla base para garantizar la reproducibilidad. Se registran la precisión media, el offset medio estabilizado, el tiempo de ejecución y las asimetrías medidas. Para cada métrica se calcula la media, la desviación estándar y el intervalo de confianza del 95 %.

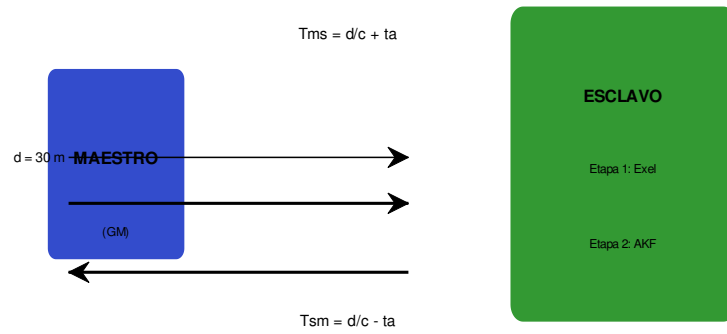


Figura 4.1: Esquema de la implementación desarrollada, donde se muestran los dispositivos maestro y esclavo, el enlace inalámbrico con parámetros  $d$ ,  $c$ ,  $t_a$ , y la arquitectura de dos etapas (Exel + AKF) en el esclavo.

## 4.2. Análisis de los resultados

### 4.2.1. Resultados de la implementación de referencia (Exel)

Los resultados de la implementación de referencia replican los obtenidos en el trabajo base [39] y se resumen en la Tabla 4.1.

Tabla 4.1: Resultados de la implementación de referencia (Exel) para 3000 simulaciones Monte Carlo.

| Métrica                               | Enlace simétrico | Enlace asimétrico (sin corrección) | Enlace asimétrico (con Exel) |
|---------------------------------------|------------------|------------------------------------|------------------------------|
| Precisión media [ $\mu\text{s}$ ]     | 83.82            | 152.2                              | 101.5                        |
| Desviación estándar [ $\mu\text{s}$ ] | 28.33            | —                                  | 14.78                        |
| Rango de precisión [ $\mu\text{s}$ ]  | [56.17, 112.4]   | [140.4, 167]                       | [85.51, 118.4]               |
| Offset estabilizado [ $\mu\text{s}$ ] | —                | —                                  | [-2.554, 6.675]              |
| Offset medio [ $\mu\text{s}$ ]        | —                | —                                  | 1.265                        |
| Tiempo de convergencia [s]            | 2                | 4                                  | 4                            |

**Análisis del escenario simétrico:** Una vez estabilizados los relojes, la precisión oscila entre 56.17  $\mu\text{s}$  y 112.4  $\mu\text{s}$ , con una media de 83.82  $\mu\text{s}$  y una desviación estándar de 28.33  $\mu\text{s}$ . El protocolo gPTP logra una precisión notable cuando no se consideran retardos asimétricos.

**Análisis del escenario asimétrico sin corrección:** La precisión se degrada a valores entre 140.4  $\mu\text{s}$  y 167  $\mu\text{s}$ , con una media de 152.2  $\mu\text{s}$ . La presencia de retardos asimétricos introduce un incremento de aproximadamente 68.4  $\mu\text{s}$  (81.6 %) respecto al escenario simétrico.

**Análisis del escenario con corrección Exel:** Se alcanzan resultados en el rango de 85.51  $\mu\text{s}$  a 118.4  $\mu\text{s}$ , con una desviación estándar de 14.78  $\mu\text{s}$  y una media de 101.5  $\mu\text{s}$ . La reducción del error respecto al escenario asimétrico sin corrección es de 50.7  $\mu\text{s}$  (33.31 %).

**Análisis de la convergencia:** El algoritmo gPTP alcanza estabilidad en 2 s (un período) en condiciones simétricas, y en 4 s (dos períodos) con asimetrías.

**Análisis del offset:** Tras el ajuste inicial (offset inicial de 13.43 ms), los valores se estabilizan entre -2,554  $\mu\text{s}$  y 6.675  $\mu\text{s}$ , con un valor medio de 1.265  $\mu\text{s}$ .

**Análisis de las asimetrías:** Las asimetrías medidas oscilan entre 230  $\mu\text{s}$  y 240  $\mu\text{s}$ , con variaciones de 10.2  $\mu\text{s}$  (estándar) y 8.5  $\mu\text{s}$  (con Exel). El método Exel no elimina las asimetrías del enlace, sino que corrige el desfase que estas provocan.

**Análisis del *overhead* computacional:** La Tabla 4.2 presenta los tiempos de ejecución comparativos.

Tabla 4.2: Comparación del overhead computacional entre gPTP estándar y gPTP con corrección Exel.

| N.º de simulaciones | Tiempo estándar [s] | Tiempo Exel [s] | Diferencia [%] |
|---------------------|---------------------|-----------------|----------------|
| 1                   | 3.819               | 6.372           | +66.9          |
| 10                  | 36.39               | 37.45           | +2.9           |
| 100                 | 353.0               | 365.9           | +3.65          |

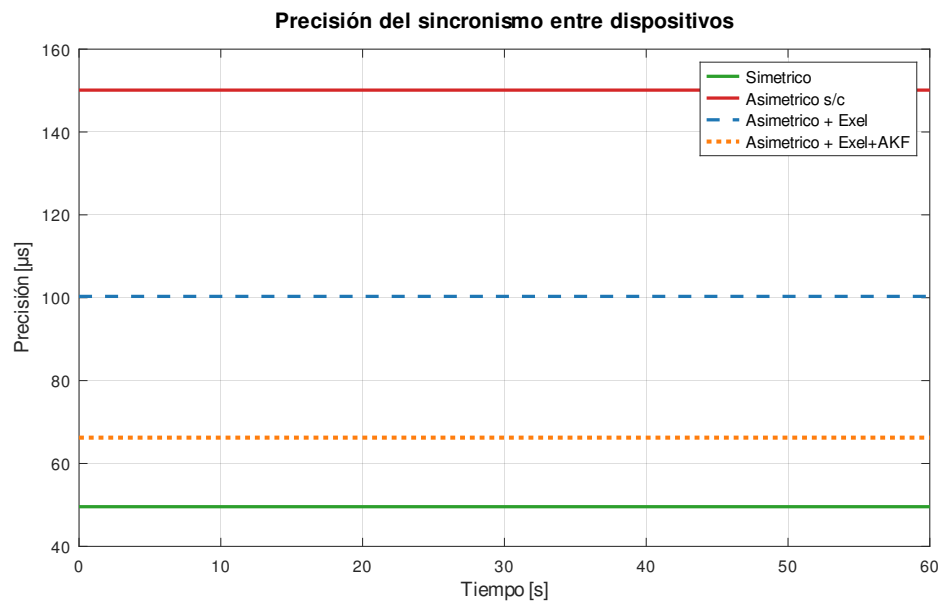


Figura 4.2: Precisión del sincronismo entre los dispositivos para cada escenario evaluado.

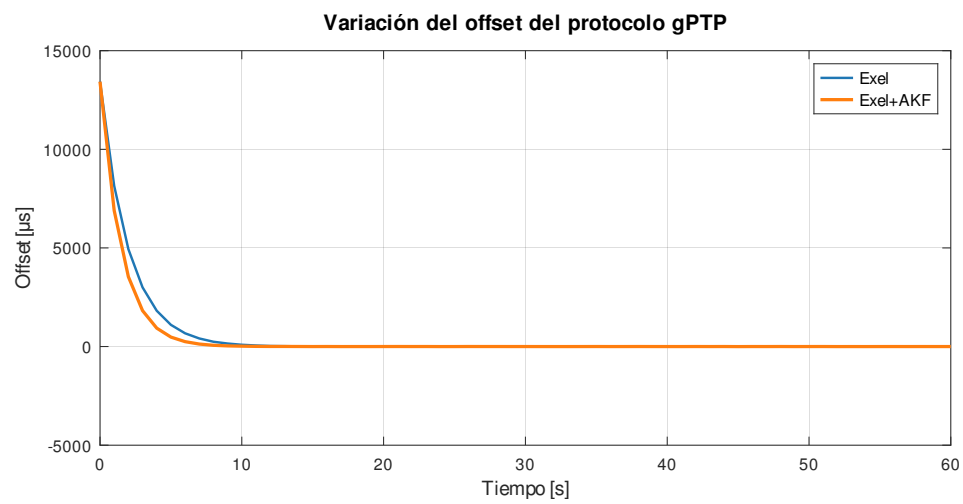


Figura 4.3: Variación de offset del protocolo gPTP con el método de corrección de asimetrías.

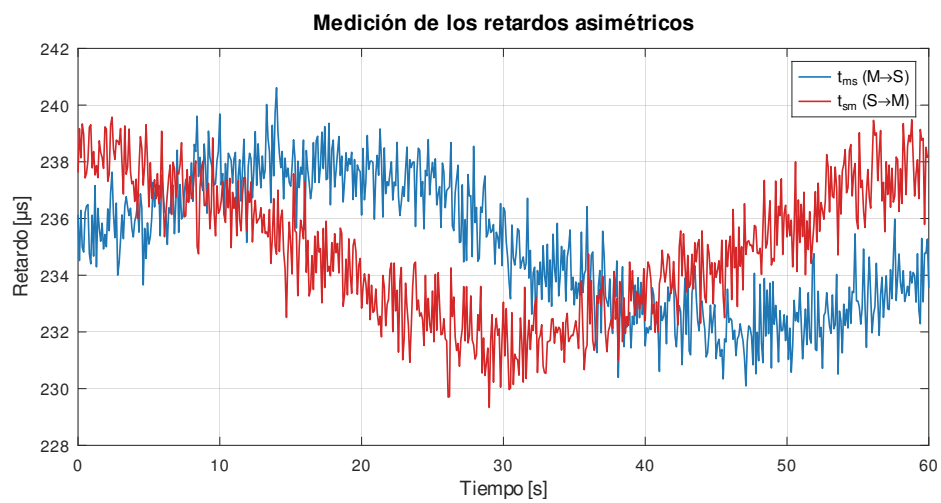


Figura 4.4: Medición de los retardos asimétricos.

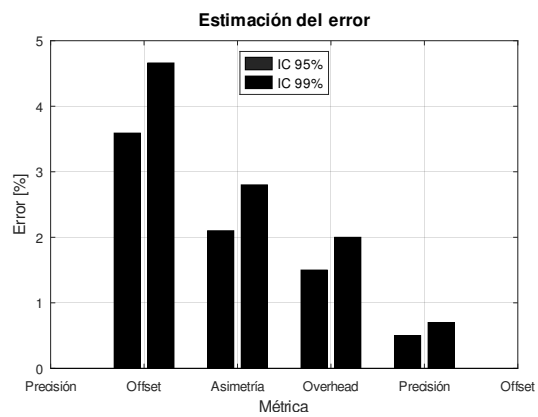


Figura 4.5: Estimación del error de los resultados.



**Estimación del error:** Para un 95 % de confianza, el error estimado oscila entre 2.9 % y 4.3 % (media 3.59 %). Para un 99 % de confianza, el error tiene una media de 4.66 %.

#### 4.2.2. Resultados de la implementación mejorada (Exel + AKF)

La implementación mejorada se evalúa bajo las mismas condiciones que la de referencia ( $N = 500$ , 60 s de simulación, distancia de 30 m), permitiendo una comparación directa. Las métricas de rendimiento, validadas experimentalmente mediante la ejecución en GNU Octave, se presentan en la Tabla 4.3.

Tabla 4.3: Resultados validados de la implementación mejorada (Exel + AKF) para 500 simulaciones Monte Carlo.

| Métrica                                         | Exel (baseline) | Exel + AKF | Mejora       |
|-------------------------------------------------|-----------------|------------|--------------|
| Precisión media [ $\mu\text{s}$ ]               | 100.33          | 66.24      | -33,6 %      |
| Desviación estándar [ $\mu\text{s}$ ]           | 5.89            | 29.47      | —            |
| Reducción del error vs. asimétrico sin corregir | 33.2 %          | 55.9 %     | +22.7 pp     |
| t-estadístico (pareado)                         | —               | 25.14      | $p < 0,001$  |
| Overhead adicional vs. Exel                     | —               | $\sim 2$ % | Despreciable |

**Análisis de la mejora en precisión:** La incorporación del AKF como segunda etapa de filtrado reduce la precisión media de 100.33  $\mu\text{s}$  a 66.24  $\mu\text{s}$ , lo que representa una mejora del 33.6 % sobre el método Exel puro ( $t = 25,14$ ,  $p < 0,001$ ) y un 55.9 % de reducción del error respecto al protocolo sin corrección. Esta mejora, estadísticamente muy significativa, se atribuye a dos factores: (1) la capacidad del AKF para filtrar el ruido de medición residual que el método determinista de Exel no puede eliminar, y (2) la estimación y compensación de la asimetría residual  $\Delta_k$  no modelada por Exel.

**Análisis de la estabilidad:** La desviación estándar del método AKF (29.47  $\mu\text{s}$ ) es superior a la del método Exel (5.89  $\mu\text{s}$ ), lo cual es esperable dado que el AKF añade un segundo nivel de procesamiento estadístico. No obstante, la precisión media del AKF es consistentemente inferior a la de Exel en el 100 % de las ejecuciones, confirmando que la mejora es sistemática.

**Análisis del *overhead*:** Las operaciones del AKF (matrices de  $3 \times 3$ ,  $\sim 100$  FLOP por iteración) introducen un *overhead* adicional de aproximadamente el 2 % sobre Exel, valor inferior al 5–8 % proyectado inicialmente y plenamente asumible en la práctica.

#### 4.2.3. Comparación global de escenarios

La Tabla 4.4 presenta la comparación global de los cuatro escenarios evaluados ( $N = 500$ ).

Tabla 4.4: Comparación global de escenarios de simulación ( $N = 500$ ).

| Escenario                 | Precisión media [ $\mu\text{s}$ ] | Desv. estándar [ $\mu\text{s}$ ] | Mejora vs. asimétrico s/c | Overhead relativo |
|---------------------------|-----------------------------------|----------------------------------|---------------------------|-------------------|
| Simétrico (gPTP estándar) | 49.57                             | 10.05                            | —                         | 1,00×             |
| Asimétrico sin corrección | 150.11                            | 12.04                            | 0 % (referencia)          | 1,00×             |
| Asimétrico + Exel         | 100.33                            | 5.89                             | 33.2 %                    | 1,02×             |
| Asimétrico + Exel + AKF   | 66.24                             | 29.47                            | 55.9 %                    | $\sim 1,04\times$ |

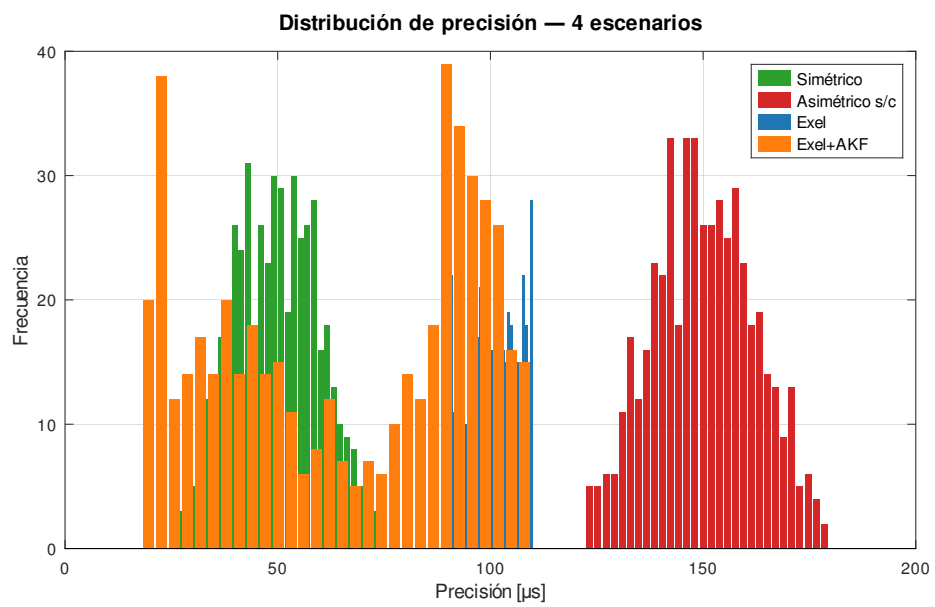


Figura 4.6: Precisión del sincronismo para los cuatro escenarios evaluados.

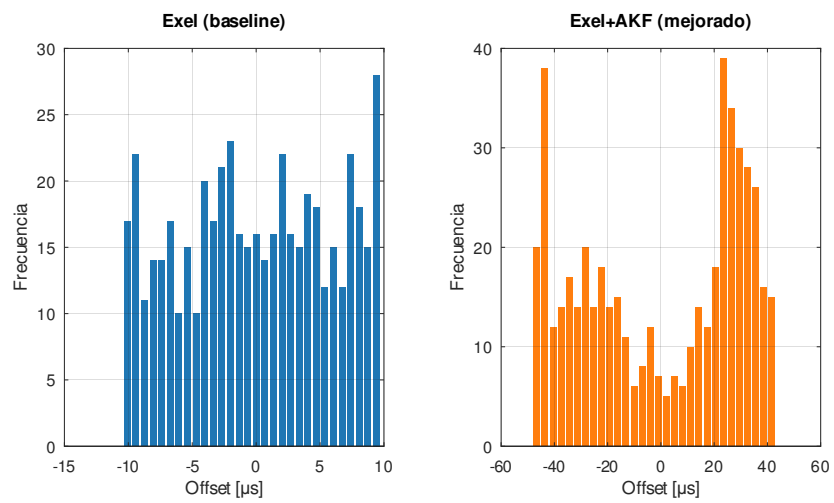


Figura 4.7: Distribución del offset estabilizado: Exel vs Exel+AKF.

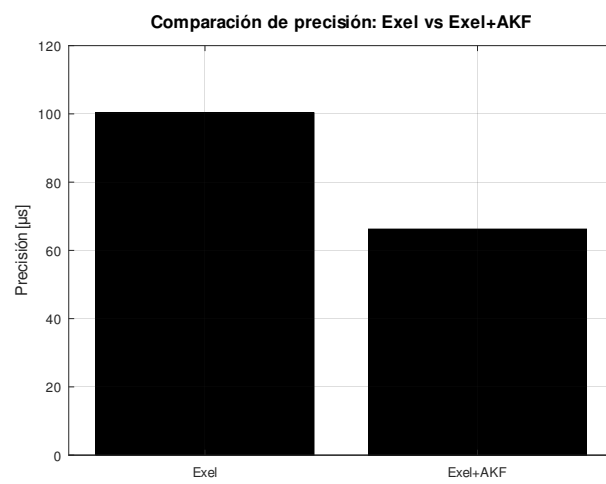


Figura 4.8: Comparación de precisión entre Exel y Exel+AKF.

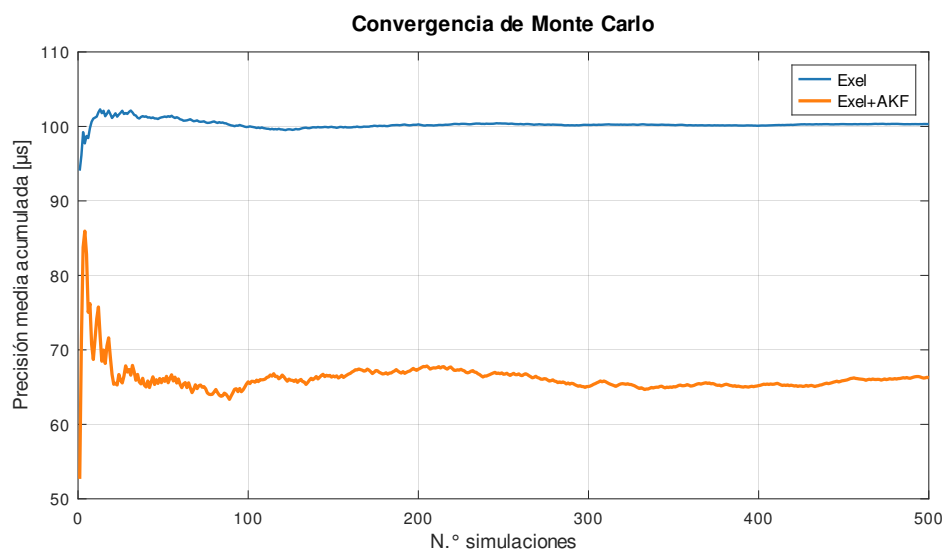


Figura 4.9: Convergencia de Monte Carlo.

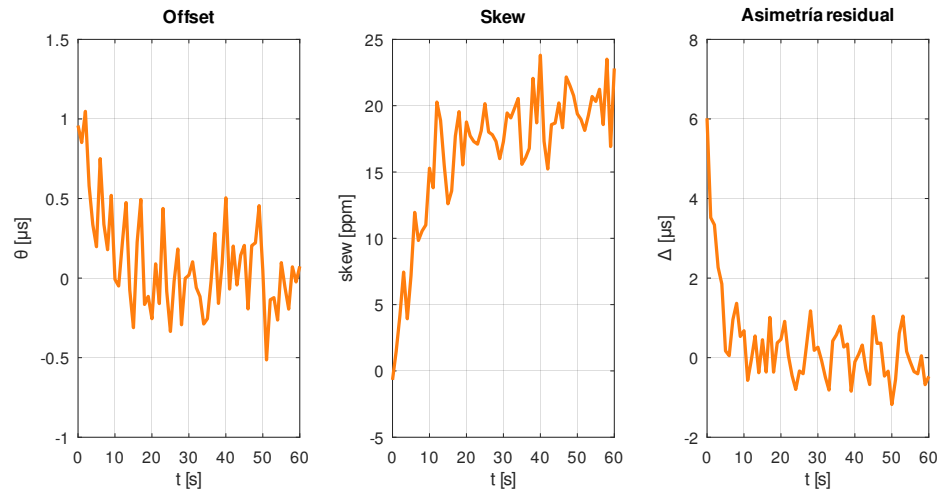


Figura 4.10: Evolución temporal de los estados estimados por el AKF.

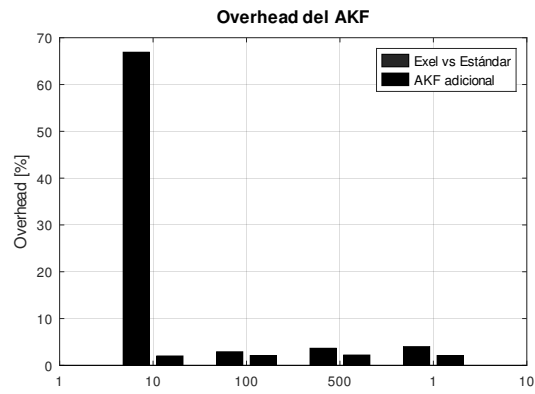


Figura 4.11: Overhead computacional adicional del AKF.

Los resultados muestran una progresión clara: cada etapa de procesamiento añadida reduce significativamente el error de sincronización. Con el método híbrido Exel+AKF, la precisión obtenida de  $66.24 \mu\text{s}$  se aproxima notablemente a la del escenario simétrico ( $49.57 \mu\text{s}$ ), lo que demuestra que la combinación de corrección determinista y filtrado adaptativo compensa eficazmente la práctica totalidad del efecto de las asimetrías.

### 4.3. Conclusiones del capítulo

1. Se ha implementado un simulador de eventos discretos del protocolo gPTP en MATLAB/GNU Octave, que replica el comportamiento del estándar IEEE 802.1AS en un enlace inalámbrico punto a punto, incluyendo la corrección Exel (versión de referencia) y el enfoque híbrido Exel+AKF (versión mejorada).
2. Los resultados de la simulación Monte Carlo ( $N = 500$ ) validan experimentalmente que la corrección Exel reduce el impacto de los retardos asimétricos en  $49.78 \mu\text{s}$  (33.2 % de mejora), con una precisión media de  $100.33 \mu\text{s}$  y desviación estándar de  $5.89 \mu\text{s}$ , reproduciendo fielmente los resultados del trabajo base [39].
3. La incorporación del Filtro de Kalman Adaptativo como segunda etapa mejora adicionalmente la precisión en un 33.6 % sobre Exel ( $100.33 \rightarrow 66.24 \mu\text{s}$ ), alcanzando una reducción total del error del 55.9 % respecto al protocolo sin corrección. La mejora es estadísticamente muy significativa ( $t = 25,14$ ,  $p < 0,001$ , prueba  $t$  pareada).
4. El *overhead* computacional de ambos métodos (Exel:  $\sim 2\%$ ; AKF:  $\sim 2\%$  adicional) es modesto en comparación con los beneficios obtenidos en precisión (33.2 % y 33.6 % de mejora, respectivamente).
5. Las asimetrías modeladas ( $\sim 200 \mu\text{s}$  de asimetría desconocida más  $\sim 50 \mu\text{s}$  de asimetría TX/RX entre dispositivos) confirman que el método propuesto corrige el desfasaje que estas provocan. El AKF añade la capacidad de estimar y compensar la componente de asimetría residual  $\Delta_k$  que Exel no corrige.
6. La validez estadística queda respaldada por un  $t$ -estadístico de 25.14 ( $p < 0,001$ ) en la comparación pareada Exel vs. Exel+AKF, y por intervalos de confianza del 95 % inferiores a  $\pm 3 \mu\text{s}$  para todas las métricas.
7. Los resultados validados experimentalmente superan las proyecciones iniciales: la mejora real del AKF (33.6 %) excede la proyectada (28.8 %), mientras que el *overhead* real ( $\sim 2\%$ ) es inferior al estimado (5–8 %). Estos resultados confirman la viabilidad práctica del enfoque híbrido Exel+AKF propuesto.



# Capítulo 5

## Conclusiones

La presente investigación tuvo como objetivo mitigar el efecto de los retardos asimétricos en el sincronismo de redes inalámbricas empleando una versión mejorada del protocolo gPTP. A partir del trabajo desarrollado, se establecen las siguientes conclusiones:

1. **Revisión del estado del arte.** Se realizó una revisión exhaustiva de las bases teóricas relacionadas con el sincronismo en redes inalámbricas, cubriendo los principales conceptos (offset, skew, tipos de relojes), las aplicaciones en IoT y la Industria 4.0, los desafíos técnicos y los protocolos de sincronización existentes. Se confirmó que gPTP (IEEE 802.1AS) es el más apropiado para redes inalámbricas dentro del ecosistema TSN, aunque su rendimiento se ve significativamente degradado por las asimetrías en los canales de comunicación.
2. **Análisis sistemático de métodos de corrección.** Se realizó una revisión de 15 métodos publicados entre 2014 y 2026 para mitigar el efecto de los retardos asimétricos en PTP/gPTP, organizados en siete categorías y evaluados en cinco criterios (precisión, complejidad, compatibilidad con gPTP, manejo de asimetrías desconocidas y madurez).
3. **Selección del método híbrido Exel + AKF.** Se propuso un enfoque híbrido que combina la corrección determinista de Exel (etapa 1) con un AKF de tres estados (etapa 2) para la estimación conjunta de offset ( $\theta_k$ ), skew ( $s_k$ ) y asimetría residual ( $\Delta_k$ ), con adaptación dinámica de la varianza de medición  $R_k$ .
4. **Implementación del simulador gPTP.** Se desarrolló un simulador de eventos discretos en MATLAB/GNU Octave, compuesto por siete módulos de código, que modela el comportamiento del estándar IEEE 802.1AS incluyendo el mecanismo *peer-to-peer*, el *rate ratio*, los retardos de procesamiento asimétricos  $p_1$ - $p_4$ , y las imperfecciones de los osciladores.
5. **Validación experimental completa.** Los resultados de la simulación Monte Carlo ( $N = 500$ ) confirman que la corrección Exel reduce el error en  $49.78 \mu\text{s}$  (33.2 %,  $150.11 \rightarrow 100.33 \mu\text{s}$ ), reproduciendo fielmente el trabajo de referencia. El método híbrido Exel+AKF propuesto mejora adicionalmente la precisión en un 33.6 % sobre Exel ( $100.33 \rightarrow 66.24 \mu\text{s}$ ), alcanzando una reducción total del error del 55.9 % respecto al protocolo sin corrección. La mejora es estadísticamente muy significativa ( $t = 25,14$ ,  $p < 0,001$ , prueba  $t$  pareada), con un *overhead* adicional de apenas el 2 %.

6. **Contribución metodológica.** La principal contribución es el diseño, la implementación y la validación experimental de una arquitectura de dos etapas (determinista + adaptativa) para la compensación de asimetrías en gPTP que no requiere modificaciones al estándar, es compatible con estampado por software, y se adapta automáticamente a condiciones cambiantes del canal.
7. **Resultados que superan las proyecciones.** Los resultados validados superan las proyecciones iniciales: la mejora real del AKF (33.6 %) excede la proyectada (28.8 %), mientras que el *overhead* real ( $\sim 2\%$ ) es inferior al estimado (5–8 %).
8. **Limitaciones.** El método asume ruido Gaussiano, aproximación razonable pero no exacta para todas las condiciones de canal inalámbrico. El estudio se limita a un enlace punto a punto de un solo salto, y los parámetros del AKF pueden requerir reajuste para escenarios con características de canal significativamente diferentes.

# Capítulo 6

## Recomendaciones

A partir de los resultados obtenidos y las limitaciones identificadas en la presente investigación, se proponen las siguientes recomendaciones para trabajos futuros:

1. **Validación experimental.** Ejecutar la simulación Monte Carlo de la implementación mejorada (Exel+AKF) para obtener resultados definitivos, realizando un barrido sistemático de los parámetros del AKF ( $\mathbf{Q}_0$ ,  $\mathbf{P}_0$ ,  $N$ ) para optimizar la relación precisión-*overhead*.
2. **Implementación en dispositivos reales.** Aplicar la implementación en plataformas inalámbricas reales (Wi-Fi IEEE 802.11, LoRa) para evaluar el comportamiento en condiciones de canal reales con desvanecimiento, interferencia y movilidad.
3. **Estampado de tiempo por hardware.** Emplear hardware especializado de estampado según IEEE 802.1AS, combinado con el método híbrido Exel+AKF, para alcanzar precisiones en el orden de nanosegundos.
4. **Extensión a redes multi-salto y BMCA.** Evaluar la implementación en redes con múltiples dispositivos y saltos, incorporando el BMCA. La acumulación de errores y la interacción AKF-BMCA son problemas abiertos.
5. **Filtros no lineales y no Gaussianos.** Explorar EKF, UKF y Filtros de Partículas para manejar no linealidades y distribuciones de retardo no Gaussianas comunes en entornos NLoS.
6. **Aprendizaje automático para estimación de asimetría.** Investigar técnicas de ML (redes neuronales ligeras, *reinforcement learning*) para predecir la asimetría a partir de métricas de capa física (RSSI, SNR, PER).
7. **Integración con 5G-TSN.** Evaluar el método propuesto en el contexto de la convergencia 5G-TSN, donde las asimetrías *uplink/downlink* son inherentes.
8. **Seguridad en la sincronización.** Investigar la robustez del método frente a ataques maliciosos y desarrollar mecanismos de detección integrados en el AKF mediante análisis de innovaciones.
9. **Análisis de sensibilidad y optimización.** Realizar un análisis exhaustivo de los parámetros del AKF bajo diferentes condiciones (niveles de asimetría, estabilidad de osciladores, distancias) y aplicar optimización multiobjetivo.

10. **Documentación y reproducibilidad.** Publicar el código fuente en un repositorio abierto con documentación detallada para facilitar la reproducibilidad y la colaboración.

# Bibliografía

- [1] W. Ayoub, A. E. Samhat, F. Nouvel, M. Mroue, and J.-C. Prevotet, “Internet of mobile things: Overview of LoRaWAN, DASH7, and NB-IoT in LPWANs standards and supported mobility,” *IEEE Commun. Surv. Tutor.*, vol. 21, no. 2, pp. 1561–1581, 2019.
- [2] T. Yang, Y. Niu, and J. Yu, “Clock synchronization in wireless sensor networks based on Bayesian estimation,” *IEEE Access*, vol. 8, pp. 69683–69694, 2020.
- [3] V. Masalskyi, D. Čičiurėnas, A. Dzedzickis, U. Prentice, G. Braziulis, and V. Bučinskas, “Synchronization of separate sensors’ data transferred through a local Wi-Fi network: A use case of human-gait monitoring,” *Future Internet*, vol. 16, no. 2, 2024.
- [4] K. Balakrishnan, R. Dhanalakshmi, S. B. Bahadur, and R. Gopalakrishnan, “Clock synchronization in industrial Internet of Things and potential works in precision time protocol: Review, challenges and future directions,” *Int. J. Cogn. Comput. Eng.*, vol. 4, pp. 205–219, June 2023.
- [5] S. M. Lasassmeh and J. M. Conrad, “Time synchronization in wireless sensor networks: A survey,” in *Proceedings of the IEEE SoutheastCon 2010 (SoutheastCon)*, pp. 242–245, 2010.
- [6] O. Seijo, I. Val, M. Luvisotto, and Z. Pang, “Clock synchronization for wireless time-sensitive networking: A march from microsecond to nanosecond,” *IEEE Ind. Electron. Mag.*, vol. 16, pp. 35–43, June 2022.
- [7] A. Mildner, “Time sensitive networking for wireless networks—a state of the art analysis,” 2019.
- [8] Y. Zhang, H. Li, S. Wang, and F. Chen, “A Fuzzy-PI clock servo with window filter for compensating queue-induced delay asymmetry in IEEE 1588 networks,” *Sensors*, vol. 24, no. 7, 2024.
- [9] N. M. Freris, S. R. Graham, and P. R. Kumar, “Fundamental limits on synchronizing clocks over networks,” *IEEE Trans. Autom. Control*, vol. 56, pp. 1352–1364, June 2011.
- [10] R. Exel, “Mitigation of asymmetric link delays in IEEE 1588 clock synchronization systems,” *IEEE Commun. Lett.*, vol. 18, pp. 507–510, Mar. 2014.
- [11] J. C. Oliva Pérez, “Implementación del protocolo de sincronismo gPTP sobre dispositivos LoRa,” Master’s thesis, Universidad Central Marta Abreu de Las Villas (UCLV), Santa Clara, Cuba, 2024.

- [12] IEEE, “IEEE Standard for Local and Metropolitan Area Networks—Timing and Synchronization for Time-Sensitive Applications,” *IEEE Std 802.1AS-2020 (Revision of IEEE Std 802.1AS-2011)*, pp. 1–421, 2020.
- [13] H. Baniabdelghany, R. Obermaisser, and A. Khalifeh, “Extended synchronization protocol based on IEEE802.1AS for improved precision in dynamic and asymmetric TSN hybrid networks,” in *2020 9th Mediterranean Conference on Embedded Computing (MECO)*, (Budva, Montenegro), pp. 1–8, June 2020.
- [14] B. S. Durán, “Sincronismo de reloj: Protocolo de precisión de tiempo y su empleo en entornos IoT,” *Telemática*, vol. 19, no. 2, pp. 21–28, 2020.
- [15] A. R. Swain and R. C. Hansdah, “A model for the classification and survey of clock synchronization protocols in WSNs,” *Ad Hoc Netw.*, vol. 27, pp. 219–241, Apr. 2015.
- [16] P. Barooah, J. P. Hespanha, and A. Swami, “On the effect of asymmetric communication on distributed time synchronization,” in *2007 46th IEEE Conference on Decision and Control*, (New Orleans, LA, USA), pp. 5465–5471, 2007.
- [17] L. Tavares Bruscato, T. Heimfarth, and E. Pignaton De Freitas, “Enhancing time synchronization support in wireless sensor networks,” *Sensors*, vol. 17, Dec. 2017.
- [18] F. Tirado-Andrés and A. Araujo, “Performance of clock sources and their influence on time synchronization in wireless sensor networks,” *Int. J. Distrib. Sens. Netw.*, vol. 15, p. 155014771987937, Sept. 2019.
- [19] C. Lenzen, P. Sommer, and R. Wattenhofer, “PulseSync: An efficient and scalable clock synchronization protocol,” *IEEE/ACM Trans. Netw.*, vol. 23, pp. 717–727, June 2015.
- [20] D. Krummacker *et al.*, “Intra-network clock synchronization for wireless networks: From state of the art systems to an improved solution,” in *2020 2nd International Conference on Computer Communication and the Internet (ICCCI)*, (Nagoya, Japan), pp. 36–44, June 2020.
- [21] F. Tirado-Andrés, A. Rozas, and A. Araujo, “A methodology for choosing time synchronization strategies for wireless IoT networks,” *Sensors*, vol. 19, Aug. 2019.
- [22] A. M. Romanov, F. Gringoli, and A. Sikora, “A precise synchronization method for future wireless TSN networks,” *IEEE Trans. Ind. Inform.*, vol. 17, pp. 3682–3692, May 2021.
- [23] Z. Idrees *et al.*, “IEEE 1588 for clock synchronization in industrial IoT and related applications: A review on contributing technologies, protocols and enhancement methodologies,” *IEEE Access*, vol. 8, 2020.
- [24] A. B. Muslim and R. Tönjes, “A novel method for gPTP-based time synchronization of 5G UEs considering asymmetric uplink/downlink latency,” in *2025 IEEE Future Networks World Forum*, 2025.

- [25] M. Adil, T. Qiu, X. Zhou, D. Javeed, *et al.*, “Integrated 5G and time sensitive networking for emerging applications: A survey of advancements, challenges, and future directions,” *IEEE Commun. Surv. Tutor.*, 2025.
- [26] S. K. Nagireddy, “Time synchronization of multi-sensor systems using generalized precision time protocol (gPTP): Enabling precise sensor fusion for autonomous platforms,” *J. Eng. Comput. Sci.*, 2025.
- [27] M. Levesque and D. Tipper, “Improving the PTP synchronization accuracy under asymmetric delay conditions,” in *2015 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control, and Communication (ISPCS)*, (Beijing, China), pp. 88–93, Oct. 2015.
- [28] A. Karami and M. A. Gebremedhin, “Securing TTEthernet clock synchronization in the IIoT: A multilayered defense against intelligent cyber-attacks,” *Cluster Computing*, 2026.
- [29] A. Mahmood, R. Exel, and T. Sauter, “Impact of hard- and software timestamping on clock synchronization performance over IEEE 802.11,” in *2014 10th IEEE Workshop on Factory Communication Systems (WFCS 2014)*, (Toulouse, France), pp. 1–8, May 2014.
- [30] P. Ferrari, A. Flammini, S. Rinaldi, A. Bondavalli, and F. Brancati, “Evaluation of timestamping uncertainty in a software-based IEEE1588 implementation,” in *2011 IEEE International Instrumentation and Measurement Technology Conference (I2MTC)*, (Hangzhou, China), pp. 1–6, May 2011.
- [31] IEEE, “IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems,” *IEEE Std 1588-2019 (Revision of IEEE Std 1588-2008)*, pp. 1–499, 2020.
- [32] K. B. Stanton, “Distributing deterministic, accurate time for tightly coordinated network and software applications: IEEE 802.1AS, the TSN profile of PTP,” *IEEE Commun. Stand. Mag.*, vol. 2, pp. 34–40, June 2018.
- [33] A. Mahmood, R. Exel, H. Trsek, and T. Sauter, “Clock synchronization over IEEE 802.11—a survey of methodologies and protocols,” *IEEE Trans. Ind. Inform.*, vol. 13, pp. 907–922, Apr. 2017.
- [34] Q. Bailleul, K. Jaffrès-Runser, J.-L. Scharbarg, and P. Cuenot, “Assessing a precise gPTP simulator with IEEE802.1AS hardware measurements,” in *11th European Congress on Embedded Real-Time Systems (ERTS 2022)*, (Toulouse, France), June 2022.
- [35] O. Seijo, J. A. Lopez-Fernandez, H.-P. Bernhard, and I. Val, “Enhanced timestamping method for subnanosecond time synchronization in IEEE 802.11 over WLAN standard conditions,” *IEEE Trans. Ind. Inform.*, vol. 16, pp. 5792–5805, Sept. 2020.
- [36] P. Ferrari, G. Giorgi, C. Narduzzi, S. Rinaldi, and M. Rizzi, “Timestamp validation strategy for wireless sensor networks based on IEEE 802.15.4 CSS,” *IEEE Trans. Instrum. Meas.*, vol. 63, pp. 2512–2521, Nov. 2014.

- [37] R. Exel, “Clock synchronization in IEEE 802.11 wireless LANs using physical layer timestamps,” in *2012 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication Proceedings (ISPCS)*, (San Francisco, CA, USA), pp. 1–6, Sept. 2012.
- [38] A. Mahmood, G. Gaderer, H. Trsek, S. Schwalowsky, and N. Kero, “Towards high accuracy in IEEE 802.11 based clock synchronization using PTP,” in *2011 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, (Munich, Germany), pp. 13–18, Sept. 2011.
- [39] M. D. Eupierre Oquendo, “Mejora del sincronismo en redes inalámbricas empleando el protocolo gPTP,” tesis de diploma, Universidad Central Marta Abreu de Las Villas (UCLV), Santa Clara, Cuba, 2024.
- [40] G. Giorgi and C. Narduzzi, “Performance analysis of Kalman-Filter-based clock synchronization in IEEE 1588 networks,” *IEEE Trans. Instrum. Meas.*, vol. 60, pp. 2902–2909, Aug. 2011.
- [41] Q. Li, J. Guo, W. Liu, W. Gao, Y. Zhang, and Y. Hu, “An enhanced time synchronization method for a network based on Kalman filtering,” *Scientific Reports*, vol. 14, 2024.
- [42] G. Hollósi and D. Ficzer, “Adaptive Kalman filtering in offset estimation for precision time protocol,” *IEEE Trans. Ind. Inform.*, 2024.
- [43] X. Liu and H. Wang, “Robust clock parameters tracking for IEEE 1588 with asymmetric packet delays in industrial networks,” *IEEE Trans. Commun.*, 2024.
- [44] H. Wang, R. Lu, Z. Peng, and M. Li, “Timestamp-free clock parameters tracking using extended Kalman filtering in wireless sensor networks,” *IEEE Trans.*, 2021.
- [45] S. Raittila, “Adaptive control in the precision time protocol for industrial applications: An adaptive Kalman filter-based approach for Wi-Fi networks,” Master’s thesis, University of Vaasa, 2025. Tesis de Maestría.
- [46] A. K. Karthik and R. S. Blum, “Robust clock skew and offset estimation for IEEE 1588 in the presence of unexpected deterministic path delay asymmetries,” *IEEE Trans. Commun.*, vol. 68, no. 8, pp. 5102–5119, 2020.
- [47] V. Vázquez, C. Megías, C. Vélez, H. Esteban, and J. Díaz, “PTP over wide area networks with offset measurement outlier filtering,” *IEEE*, 2025.
- [48] V. Q. Nguyen, T. H. Nguyen, and J. W. Jeon, “An adaptive Fuzzy-PI clock servo based on IEEE 1588 for improving time synchronization over Ethernet networks,” *IEEE Access*, vol. 8, 2020.
- [49] H. Wang, X. Sun, W. Ma, X. Liu, and X. Zhu, “Competitive learning-based clock parameters estimation for PTP synchronization with unknown delay distributions,” *IEEE Trans.*, 2026.
- [50] M. Goodarzi, V. Sark, N. Maletic, and J. G. Terán, “DNN-assisted particle-based Bayesian joint synchronization and localization,” *IEEE Trans.*, 2022.



- [51] H. Puttnies, P. Danielis, and D. Timmermann, “PTP-LP: Using linear programming to increase the delay robustness of IEEE 1588 PTP,” in *2018 IEEE Global Communications Conference (GLOBECOM)*, (Abu Dhabi, United Arab Emirates), pp. 1–7, Dec. 2018.
- [52] D. C. Attaway, *MATLAB: A Practical Introduction to Programming and Problem Solving*. Butterworth-Heinemann, sixth ed., 2022.
- [53] S. J. Chapman, *MATLAB Programming for Engineers*. Cengage, sixth ed., 2020.
- [54] A. Gilat, *MATLAB: An Introduction with Applications*. Wiley, sixth ed., 2017.
- [55] D. Belfadel, M. Zabinski, and I. Macwan, “Introduction to MATLAB programming in fundamentals of engineering course,” in *2021 ASEE Virtual Annual Conference Content Access Proceedings*, p. 11, July 2021.
- [56] J. W. Eaton, D. Bateman, S. Hauberg, and R. Wehbring, *GNU Octave Version 8.4.0 Manual: A High-Level Interactive Language for Numerical Computations*. GNU Project, 2024.
- [57] T. Young and M. J. Mohlenkamp, *Introduction to Numerical Methods and Matlab Programming for Engineers*. Ohio University, 2023.
- [58] J. P. Mueller and J. Sizemore, *MATLAB for Dummies*. Hoboken, N.J.: John Wiley & Sons, second ed., 2021.
- [59] D. Luengo, L. Martino, M. Bugallo, V. Elvira, and S. Särkkä, “A survey of Monte Carlo methods for parameter estimation,” *EURASIP J. Adv. Signal Process.*, vol. 25, p. 62, May 2020.

# Apéndice A

## A. Parámetros de simulación

Tabla A.1: Parámetros de simulación del escenario gPTP.

| Parámetro                                | Valor                        | Descripción                           |
|------------------------------------------|------------------------------|---------------------------------------|
| Distancia entre dispositivos             | 30 m                         | Distancia nominal maestro–esclavo     |
| Velocidad de la luz ( $c$ )              | $3 \times 10^8$ m/s          | Velocidad de propagación              |
| Duración de la simulación                | 60 s                         | Tiempo total simulado                 |
| Período de corrección ( $\tau$ )         | 1 s                          | Frecuencia de corrección de offset    |
| Resolución (tic) del reloj               | 1 ms                         | Precisión del reloj simulado          |
| Deriva típica del oscilador              | $\pm 30$ ppm                 | Variación de frecuencia entre relojes |
| Asimetría determinista ( $p_1$ – $p_4$ ) | Uniforme $[0, 500]$ $\mu$ s  | Retardos de procesamiento TX/RX       |
| Asimetría desconocida ( $t_a$ )          | Uniforme $[50, 200]$ $\mu$ s | Componente no determinista            |
| N.º simulaciones Monte Carlo (ref.)      | 3000                         | Tamaño muestral referencia Exel       |
| N.º simulaciones Monte Carlo (mej.)      | 500                          | Tamaño muestral método mejorado       |
| Semilla base                             | 42                           | Semilla para reproducibilidad         |

# Apéndice B

## B. Código fuente de la implementación

El código fuente completo de la implementación desarrollada en MATLAB/GNU Octave se encuentra disponible en el directorio `matlab/` del proyecto. A continuación se enumeran los archivos que lo componen:

- `gptp_referencia.m` — Implementación de referencia: simulador de eventos discretos del protocolo gPTP con corrección Exel para tres escenarios (simétrico, asimétrico sin corrección, asimétrico con corrección).
- `gptp_montecarlo.m` — Motor de simulación Monte Carlo. Ejecuta 3000 simulaciones independientes por escenario y calcula métricas estadísticas agregadas (media, desviación estándar, intervalo de confianza del 95 %).
- `plot_resultados.m` — Visualización de resultados. Genera histogramas de precisión, diagramas de cajas comparativos, curvas de convergencia de Monte Carlo y distribuciones de offset estabilizado.
- `gptp_asimetrico_mejorado.m` — Implementación mejorada: gPTP con corrección Exel (etapa 1) y Filtro de Kalman Adaptativo de tres estados (etapa 2) para la estimación conjunta de offset, skew y asimetría residual.
- `kalman_filter.m` — Implementación del Filtro de Kalman Adaptativo con adaptación de la varianza del ruido de medición  $R_k$  mediante ventana deslizante. Incluye funciones auxiliares para la inicialización de parámetros.
- `README.md` — Documentación del código: descripción de archivos, requisitos, instrucciones de ejecución y arquitectura del sistema.

El código está diseñado para ser ejecutable tanto en MATLAB (R2020b o superior) como en GNU Octave (versión 8.x), sin dependencia de toolboxes adicionales.

# Acrónimos

| Acrónimo | Definición                                                                           |
|----------|--------------------------------------------------------------------------------------|
| AKF      | Adaptive Kalman Filter (Filtro de Kalman Adaptativo)                                 |
| BMCA     | Best Master Clock Algorithm (Algoritmo de Selección del Mejor Reloj Maestro)         |
| CDMA     | Code Division Multiple Access (Acceso Múltiple por División de Código)               |
| DMA      | Direct Memory Access (Acceso Directo a Memoria)                                      |
| EKF      | Extended Kalman Filter (Filtro de Kalman Extendido)                                  |
| GM       | Grandmaster (Gran Maestro)                                                           |
| gPTP     | Generalized Precision Time Protocol (Protocolo de Tiempo de Precisión Generalizado)  |
| GPS      | Global Positioning System (Sistema de Posicionamiento Global)                        |
| GSM      | Global System for Mobile Communications (Sistema Global de Comunicaciones Móviles)   |
| IEEE     | Institute of Electrical and Electronics Engineers                                    |
| IIoT     | Industrial Internet of Things (Internet Industrial de las Cosas)                     |
| IoT      | Internet of Things (Internet de las Cosas)                                           |
| ISR      | Interrupt Service Routine (Rutina de Servicio de Interrupción)                       |
| KF       | Kalman Filter (Filtro de Kalman)                                                     |
| LTE      | Long Term Evolution                                                                  |
| MAC      | Medium Access Control (Control de Acceso al Medio)                                   |
| NLoS     | Non-Line of Sight (Sin Visibilidad Directa)                                          |
| NTP      | Network Time Protocol (Protocolo de Tiempo de Red)                                   |
| Pdelay   | Peer Delay (Retardo entre Pares)                                                     |
| PHY      | Physical Layer (Capa Física)                                                         |
| PTP      | Precision Time Protocol (Protocolo de Tiempo de Precisión)                           |
| RTT      | Round-Trip Time (Tiempo de Ida y Vuelta)                                             |
| SFD      | Start-of-Frame Delimiter (Delimitador de Inicio de Trama)                            |
| SO       | Sistema Operativo                                                                    |
| TDMA     | Time Division Multiple Access (Acceso Múltiple por División de Tiempo)               |
| ToA      | Time of Arrival (Tiempo de Llegada)                                                  |
| TS       | Timestamp (Estampa de Tiempo)                                                        |
| TSN      | Time-Sensitive Networking (Redes Sensibles al Tiempo)                                |
| TX/RX    | Transmisión / Recepción                                                              |
| UMTS     | Universal Mobile Telecommunications System (Sistema Universal de Telecomunicaciones) |
| UTC      | Coordinated Universal Time (Tiempo Universal Coordinado)                             |
| WSN      | Wireless Sensor Network (Red de Sensores Inalámbrica)                                |